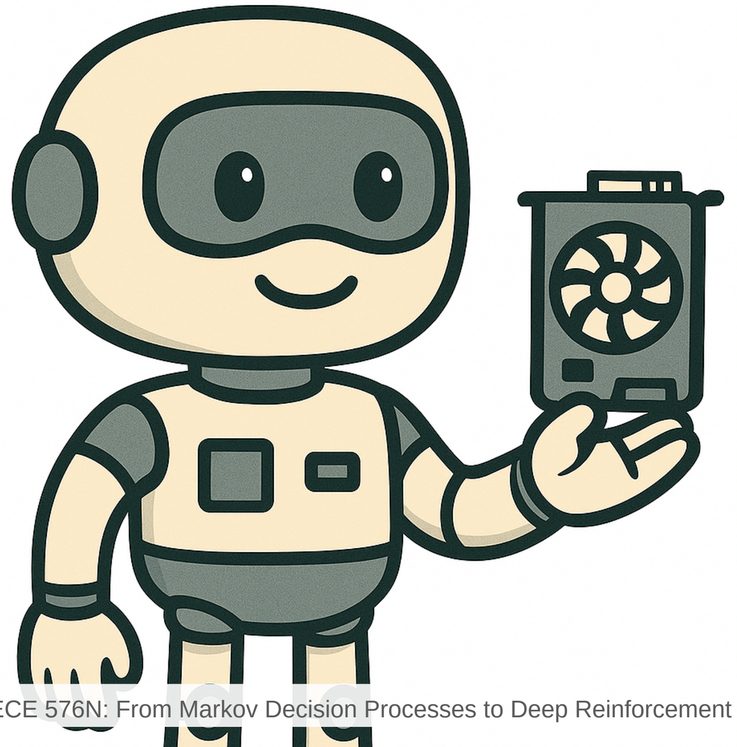
# L10: Function approximation in RL

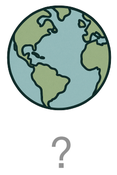
EECE 576N | From MDPs to Deep RL | Fall 2026

Cyrus Neary | cyrus.neary@ubc.ca



# What is reinforcement learning?

How can we solve sequential decision-making problems without a model of the environment?



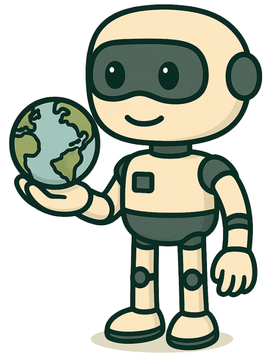
$$\mathscr{M} = (S, A, T, R, \gamma, \mu, \Omega, O)$$

? ? ? ? ? ? ?

Some or all of the (PO)MDP components are unknown.

The answer: learn from interactions with the environment.

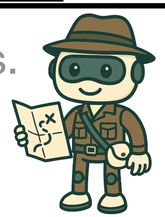
More specifically: Design an agent that gathers data from the environment and uses it to estimate $\pi^*$, $V^*(s)$, $T(s' \mid , s, a)$, $R(s, a)$, and other quantities.



How do we explore the unknown environment to gather useful experience for learning? How do we balance exploration with exploitation?

Exploration problems.

**Last class.**



How do we use our experience to update approximations of the value function and optimal policy?

Fundamental RL algorithms.

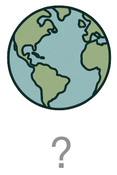
**Last few lectures.**

How do we build expressive function classes to help us learn and make decisions from diverse observation (e.g., images, language, etc.)?

Deep RL algorithms.

# What is reinforcement learning?

How can we solve sequential decision-making problems without a model of the environment?



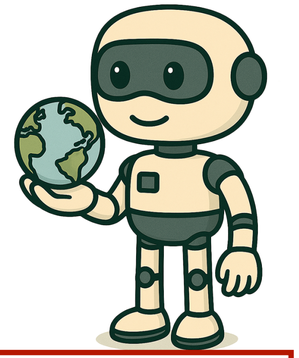
$$\mathscr{M} = (S, A, T, R, \gamma, \mu, \Omega, O)$$

? ? ? ? ? ?

Some or all of the (PO)MDP components are unknown.

The answer: learn from interactions with the environment.

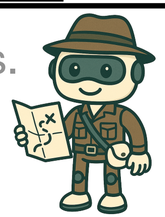
More specifically: Design an agent that gathers data from the environment and uses it to estimate $\pi^*$, $V^*(s)$, $T(s' \mid , s, a)$, $R(s, a)$, and other quantities.



How do we explore the unknown environment to gather useful experience for learning? How do we balance exploration with exploitation?

Exploration problems.

**Last class.**



How do we use our experience to update approximations of the value function and optimal policy?

Fundamental RL algorithms.

**Last few lectures.**

How do we build expressive function classes to help us learn and make decisions from diverse observation (e.g., images, language, etc.)?

Deep RL algorithms.

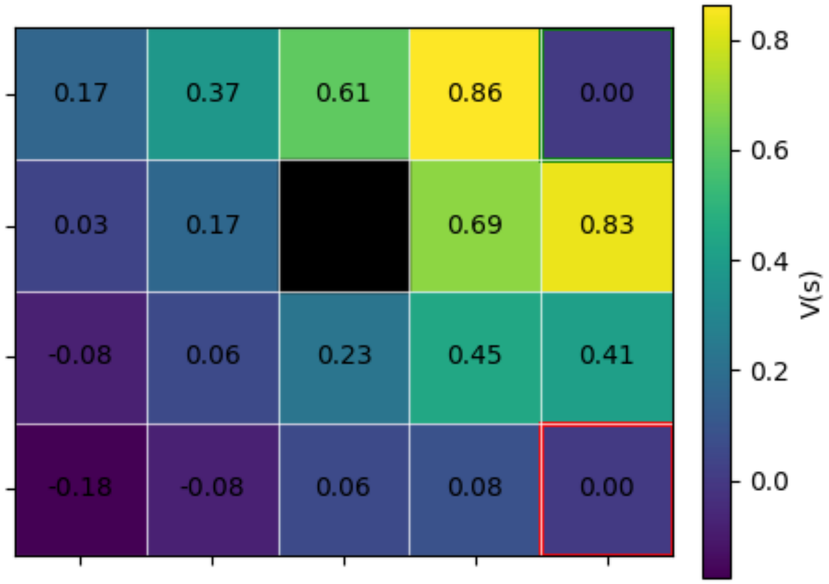
**Next few lectures.**

# Function approximation in RL

Key idea: replace the tables used to represent value $V$ and policy $\pi$ functions, with parameterized functions $V_\theta$, $\pi_\theta$ for some paramer vector(s) $\theta \in \mathbb{R}^d$.

## Tabular RL

$V(s) =$



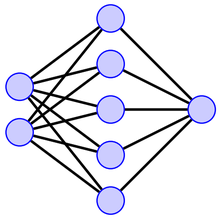

## RL with function approximation

$V_\theta(s) =$ Parametric function approximator.

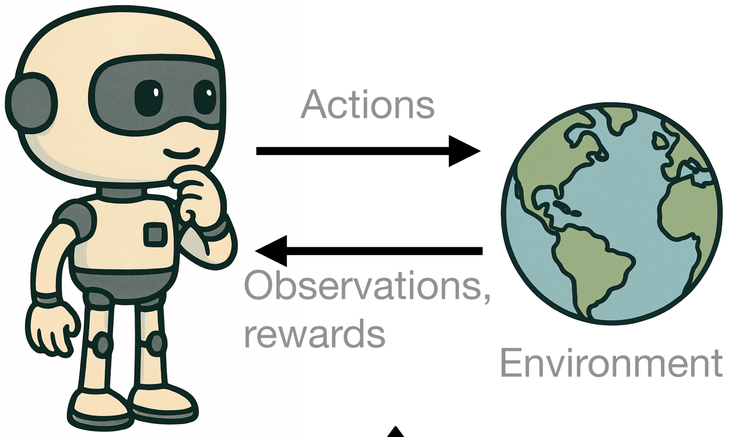
e.g.,

Linear models: $V_\theta(s) = \theta^T \phi(s)$, for feature vectors $\phi(s) \in \mathbb{R}^d$

Neural networks: $V_\theta(s) = NN_\theta(s)$, each layer extracts higher-level features, and can approximate continuous functions.



# Generalized policy iteration




$$\tau^1 = s_0^1, a_0^1, r_0^1, s_1^1, a_1^1, r_1^1, \ldots, s_{T_1}^1$$

$$\tau^2 = s_0^2, a_0^2, r_0^2, s_1^2, a_1^2, r_1^2, \ldots, s_{T_2}^2$$

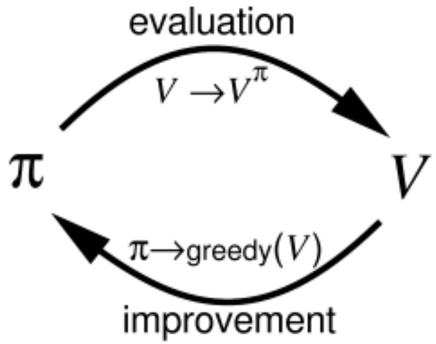



Figure courtesy of Sutton & Barto (2018)

Use estimate of $Q_\theta(s, a)$ to improve $\pi \to \pi'$

Learned parameterized estimate $Q_\theta(s, a)$

# Why is function approximation useful in RL?

- Approximate continuous functions easily.
- Can automatically differentiate through NNs to define training algorithms.
- Like with tabular case, we need to learn because it's hard to write down $V(s)$ and $\pi(a|s)$ from first principles.
- Leverage representation learning capabilities to learn good features for value/policy estimation from diverse/high-dimensional observations.
- Policy is now a first-class citizen instead of being defined in terms of value.
- Necessary to move beyond tabular domains. As $|\mathcal{S}|$ grows we can't afford to keep tracte of every $V(s)$ individually, instead learn $V_\theta(s)$, where $\theta \in \mathbb{R}^d$, $d \ll |\mathcal{S}|$.
  - ↳ only possible because we expect our function $V_\theta(s)$ to generalize between states.
- Deal with partial observations by learning latent features from raw observations.

# (A whirlwind intro to) Function approximation

Broadly, represent an unknown or intractable function with a another, usually simpler, one.

- Numerical analysis: approximating $u(x)$ that satisfies differential equation using basis expansions or piecewise polynomails.
- Approximation theory: study of *how well* certain function classes (e.g., polynomials, splines) can approximate continuous functions.
- Symbolic or analytical methods: Taylor series, Fourier series, finite elements.

# (A whirlwind intro to) Function approximation

Broadly, represent an unknown or intractable function with a another, usually simpler, one.

- Numerical analysis: approximating $u(x)$ that satisfies differential equation using basis expansions or piecewise polynomails.
- Approximation theory: study of *how well* certain function classes (e.g., polynomials, splines) can approximate continuous functions.
- Symbolic or analytical methods: Taylor series, Fourier series, finite elements.

In this class, we'll focus on learning *parametric* function approximations from data:

$$f(x) \approx f_\theta(x) \qquad \text{with parameter vector } \theta \in \mathbb{R}^d$$

I.e., assume a form of the function, and only change the parameter vector $\theta$ to try and match $f(x)$.

# (A whirlwind intro to) Function approximation

## Linear function approximation

Suppose we have pre-defined basis functions $\phi(\cdot)$ mapping inputs $x$ to feature vectors $\phi(x) \in \mathbb{R}^d$.

Define $\hat{f}(x;\theta)$ as a weighted combination of the feature vector's elements.

$$\hat{f}_\theta(x) = \theta^T \phi(x)$$

# (A whirlwind intro to) Function approximation

## Linear function approximation

Suppose we have pre-defined basis functions $\phi(\cdot)$ mapping inputs $x$ to feature vectors $\phi(x) \in \mathbb{R}^d$.

Define $\hat{f}(x;\theta)$ as a weighted combination of the feature vector's elements.

$$\hat{f}_\theta(x) = \theta^T \phi(x)$$

## Neural networks as function approximators

At this point, there are *many* different types of neural networks.

E.g., transformers, convolutional neural networks, graph neural networks, feedforward neural networks.

# (A whirlwind intro to) Function approximation

## Linear function approximation

Suppose we have pre-defined basis functions $\phi(\cdot)$ mapping inputs $x$ to feature vectors $\phi(x) \in \mathbb{R}^d$.

Define $\hat{f}(x;\theta)$ as a weighted combination of the feature vector's elements.

$$\hat{f}_\theta(x) = \theta^T \phi(x)$$

## Neural networks as function approximators

At this point, there are *many* different types of neural networks.

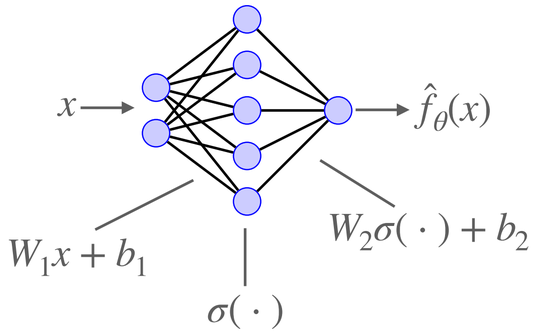
E.g., transformers, convolutional neural networks, graph neural networks, feedforward neural networks.

Basic *multi-layer perceptrons* generalize linear models by stacking linear transformations and non-linear "activation functions".

e.g., 2-layer network:

$$\hat{f}_\theta(x) = W_2\sigma\left(W_1 x + b_1\right) + b_2$$

$$\theta = (W_2, b_2, W_1, b_1)$$



# (A whirlwind intro to) Supervised learning

Setting: We would like to learn a function $f : X \to Y$ that outputs $y \in Y$ given an input $x \in X$.

Assume there exists a joint probability distribution $\mathscr{D}$ over $(x, y)$,
and define a loss function $\mathscr{L} : Y \times Y \to \mathbb{R}$ defining the “difference” between a predicted output $\hat{y}$ and the true label $y$.

# (A whirlwind intro to) Supervised learning

Setting: We would like to learn a function $f : X \to Y$ that outputs $y \in Y$ given an input $x \in X$.

Assume there exists a joint probability distribution $\mathscr{D}$ over $(x, y)$,
and define a loss function $\mathscr{L} : Y \times Y \to \mathbb{R}$ defining the "difference" between a predicted output $\hat{y}$ and the true label $y$.

E.g., in regression tasks, a common loss function is the squared distance $\mathscr{L}(\hat{y}, y) = ||\hat{y} - y||^2$.

# (A whirlwind intro to) Supervised learning

Setting: We would like to learn a function $f : X \to Y$ that outputs $y \in Y$ given an input $x \in X$.

Assume there exists a joint probability distribution $\mathscr{D}$ over $(x, y)$,
and define a loss function $\mathscr{L} : Y \times Y \to \mathbb{R}$ defining the "difference" between a predicted output $\hat{y}$ and the true label $y$.

E.g., in regression tasks, a common loss function is the squared distance $\mathscr{L}(\hat{y}, y) = ||\hat{y} - y||^2$.

Objective: We want to minimize the **risk** $R(f)$ of a hypothesis function $f(x)$ coming from some hypothesis class $\mathscr{H}$.

$$R(f) = \mathbb{E}_{(x,y)\sim\mathscr{D}}\left[\mathscr{L}(f(x), y)\right] \qquad f^* \in \arg\min_{f\in\mathscr{H}} R(f)$$

# (A whirlwind intro to) Supervised learning

Setting: We would like to learn a function $f : X \to Y$ that outputs $y \in Y$ given an input $x \in X$.

Assume there exists a joint probability distribution $\mathscr{D}$ over $(x, y)$,
and define a loss function $\mathscr{L} : Y \times Y \to \mathbb{R}$ defining the "difference" between a predicted output $\hat{y}$ and the true label $y$.

E.g., in regression tasks, a common loss function is the squared distance $\mathscr{L}(\hat{y}, y) = ||\hat{y} - y||^2$.

Objective: We want to minimize the **risk** $R(f)$ of a hypothesis function $f(x)$ coming from some hypothesis class $\mathscr{H}$.

$$R(f) = \mathbb{E}_{(x,y)\sim\mathscr{D}}\left[\mathscr{L}(f(x), y)\right] \qquad f^* \in \arg\min_{f\in\mathscr{H}} R(f)$$

e.g., if our hypothesis class $\mathscr{H}$ is a particular collection of parameterized functions, then we want $\theta^* \in \arg\min_{\theta\in\mathbb{R}^d} R(f_\theta)$.

# (A whirlwind intro to) Supervised learning

Setting: We would like to learn a function $f : X \to Y$ that outputs $y \in Y$ given an input $x \in X$.

Assume there exists a joint probability distribution $\mathscr{D}$ over $(x, y)$,
and define a loss function $\mathscr{L} : Y \times Y \to \mathbb{R}$ defining the "difference" between a predicted output $\hat{y}$ and the true label $y$.

E.g., in regression tasks, a common loss function is the squared distance $\mathscr{L}(\hat{y}, y) = ||\hat{y} - y||^2$.

Objective: We want to minimize the **risk** $R(f)$ of a hypothesis function $f(x)$ coming from some hypothesis class $\mathscr{H}$.

$$R(f) = \mathbb{E}_{(x,y)\sim\mathscr{D}}\left[\mathscr{L}(f(x), y)\right] \qquad f^* \in \arg\min_{f\in\mathscr{H}} R(f)$$

e.g., if our hypothesis class $\mathscr{H}$ is a particular collection of parameterized functions, then we want $\theta^* \in \arg\min_{\theta\in\mathbb{R}^d} R(f_\theta)$.

In general, we don't know $\mathscr{D}$ and can't compute $R(f)$. Instead, given a dataset of $N$ i.i.d. samples $(x_i, y_i) \sim \mathscr{D}$,
we aim to minimize the **empirical risk** $\hat{R}(f)$**:**

$$\hat{R}(f) = \frac{1}{N}\sum_{i=1}^{N} \mathscr{L}(f(x_i), y_i) \qquad f^* = \arg\min_{f\in\mathscr{H}} \hat{R}(f)$$

# (Stochastic) gradient descent

Given a hypothesis class of parameterized functions $f_\theta \in \mathscr{H}$, and a fixed dataset of $N$ i.i.d. samples $(x_i, y_i) \sim \mathscr{D}$, we aim to solve the empirical risk minimization problem:

$$\theta^* \in \arg\min_{\theta \in \mathbb{R}^d} \frac{1}{N} \sum_{i=1}^{N} \mathscr{L}(f_\theta(x_i), y_i)$$

# (Stochastic) gradient descent

Given a hypothesis class of parameterized functions $f_\theta \in \mathscr{H}$, and a fixed dataset of $N$ i.i.d. samples $(x_i, y_i) \sim \mathscr{D}$, we aim to solve the empirical risk minimization problem:

$$\theta^* \in \arg\min_{\theta \in \mathbb{R}^d} \frac{1}{N} \sum_{i=1}^{N} \mathscr{L}(f_\theta(x_i), y_i)$$

Gradient descent is one (very popular and universal) algorithm for finding local minima of such optimization problems.

Compute $\nabla_\theta \hat{R}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \nabla_\theta \mathscr{L}(f_\theta(x_i), y_i)$

And iteratively take steps in the direction of steepest descent: $\theta_{t+1} = \theta_t - \alpha \nabla_\theta \hat{R}(\theta)$

Note: this technique for *batch* gradient descent requires we compute $\nabla_\theta \mathscr{L}(f_\theta(x_i), y_i)$ for **every** sample $(x_i, y_i)$ in the training datset.

# (Stochastic) gradient descent

Given a hypothesis class of parameterized functions $f_\theta \in \mathscr{H}$, and a fixed dataset of $N$ i.i.d. samples $(x_i, y_i) \sim \mathscr{D}$, we aim to solve the empirical risk minimization problem:

$$\theta^* \in \arg\min_{\theta \in \mathbb{R}^d} \frac{1}{N} \sum_{i=1}^{N} \mathscr{L}(f_\theta(x_i), y_i)$$

Gradient descent is one (very popular and universal) algorithm for finding local minima of such optimization problems.

Compute $\nabla_\theta \hat{R}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \nabla_\theta \mathscr{L}(f_\theta(x_i), y_i)$

And iteratively take steps in the direction of steepest descent: $\theta_{t+1} = \theta_t - \alpha \nabla_\theta \hat{R}(\theta)$

Note: this technique for *batch* gradient descent requires we compute $\nabla_\theta \mathscr{L}(f_\theta(x_i), y_i)$ for **every** sample $(x_i, y_i)$ in the training datset.

Stochastic gradient descent doesn't use the whole dataset at every gradient step. Instead, sample one (or a few) data points uniformly from the dataset, and use an *unbiased estimator* of the gradient of $\nabla_\theta \hat{R}(\theta)$.

Sample $g_t = \nabla_\theta \mathscr{L}(f_\theta(x_i), y_i)$, with $i_t \sim \text{Uniform}(\{1, \ldots, N\})$

And update $\theta_{t+1} = \theta_t - \alpha g_t$

In expectation, $\mathbb{E}[g_t] = \nabla_\theta \hat{R}(\theta)$

# How is RL different from supervised learning? How could this affect architectures primarily developed for supervised learning?

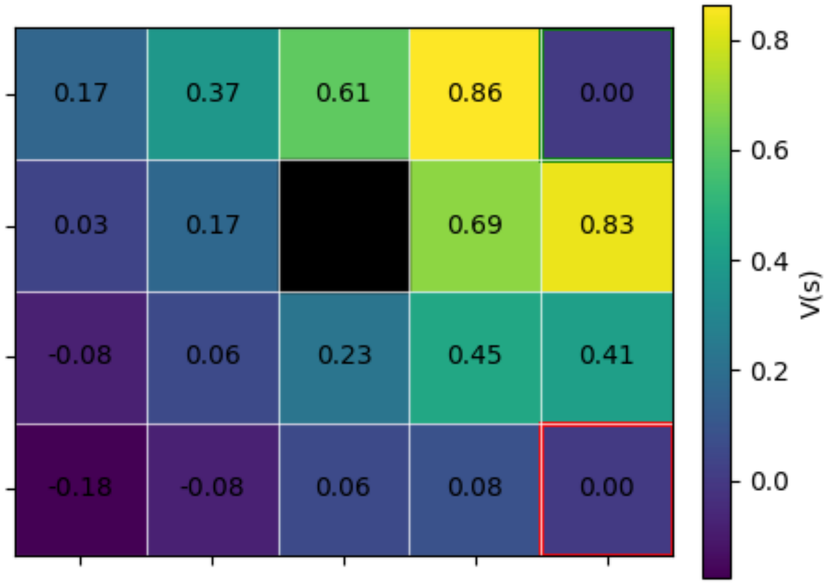
- Supervised learning: We have the target (i.e. labels $y$)
  - in RL, we need to use estimates of the labels/targets, which may be imperfect.
- RL interacts with an environment and learns in the context of its (implicitly defined) constraints.
- Supervised learning – minimize risk
  RL – maximize future discounted rewards.
- RL interacts with environment, the policy we learn changes → moving target as we update $\pi$.
  - SL, → static dataset that we can iterate over many times using SGD.
- "End-to-end" RL training requires tuning several difficult "sub-problems" e.g. feature extraction, map features to actions and values, all with one objective defined by optimizing reward.
- Bootstrapping + function approximation → trouble ☹

# (Back to) function approximation

Key idea: replace the tables used to represent value $V$ and policy $\pi$ functions, with parameterized functions $V_\theta$, $\pi_\theta$ for some paramer vector(s) $\theta \in \mathbb{R}^d$.

## Tabular RL

$V(s) =$



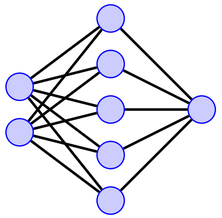

## RL with function approximation

$V_\theta(s) =$ Parametric function approximator.

e.g.,

Linear models: $V_\theta(s) = \theta^T \phi(s)$, for feature vectors $\phi(s) \in \mathbb{R}^d$

Neural networks: $V_\theta(s) = NN_\theta(s)$, each layer extracts higher-level features, and can approximate continuous functions.



# Learning the value function from on-policy data

What objective should we use to learn an approximation of the value function?

$$R(\theta) = \mathbb{E}_{s\sim\mu}\left[\left(V^{\pi}(s) - V_{\theta}(s)\right)^{2}\right] = \sum_{s\in S}\mu(s)\left(V^{\pi}(s) - V_{\theta}(s)\right)^{2}$$

# Learning the value function from on-policy data

What objective should we use to learn an approximation of the value function?

$$R(\theta) = \mathbb{E}_{s\sim\mu}\left[\left(V^{\pi}(s) - V_{\theta}(s)\right)^2\right] = \sum_{s\in S}\mu(s)\left(V^{\pi}(s) - V_{\theta}(s)\right)^2$$

Some distribution over states, weighting their importance

A common choice: The distribution over states induced by the policy $\pi$, with the intuition that we care about more heavily weighting the error for states that we frequently visit under the current policy.

# Learning the value function from on-policy data

What objective should we use to learn an approximation of the value function?

The loss function $\mathscr{L}(\hat{y}, y) = (y - \hat{y})^2$

$$R(\theta) = \mathbb{E}_{s \sim \mu}\left[\left(V^{\pi}(s) - V_{\theta}(s)\right)^2\right] = \sum_{s \in S} \mu(s)\left(V^{\pi}(s) - V_{\theta}(s)\right)^2$$

Some distribution over states, weighting their importance

A common choice: The distribution over states induced by the policy $\pi$, with the intuition that we care about more heavily weighting the error for states that we frequently visit under the current policy.

# Learning the value function from on-policy data

## What objective should we use to learn an approximation of the value function?

The loss function $\mathscr{L}(\hat{y}, y) = (y - \hat{y})^2$

The true value function to be estimated

$$R(\theta) = \mathbb{E}_{s\sim\mu}\left[\left(V^{\pi}(s) - V_{\theta}(s)\right)^2\right] = \sum_{s\in S} \mu(s)\left(V^{\pi}(s) - V_{\theta}(s)\right)^2$$

Some distribution over states, weighting their importance

A common choice: The distribution over states induced by the policy $\pi$, with the intuition that we care about more heavily weighting the error for states that we frequently visit under the current policy.

# Learning the value function from on-policy data

## What objective should we use to learn an approximation of the value function?

The loss function $\mathscr{L}(\hat{y}, y) = (y - \hat{y})^2$

The true value function to be estimated

$$R(\theta) = \mathbb{E}_{s\sim\mu}\left[\left(V^{\pi}(s) - V_{\theta}(s)\right)^2\right] = \sum_{s\in S} \mu(s)\left(V^{\pi}(s) - V_{\theta}(s)\right)^2$$

Some distribution over states, weighting their importance

The parameterized value function approximation

A common choice: The distribution over states induced by the policy $\pi$, with the intuition that we care about more heavily weighting the error for states that we frequently visit under the current policy.

# Learning the value function from on-policy data

What objective should we use to learn an approximation of the value function?

The loss function $\mathscr{L}(\hat{y}, y) = (y - \hat{y})^2$

The true value function to be estimated

$$R(\theta) = \mathbb{E}_{s \sim \mu}\left[\left(V^{\pi}(s) - V_{\theta}(s)\right)^2\right] = \sum_{s \in S} \mu(s)\left(V^{\pi}(s) - V_{\theta}(s)\right)^2$$

Some distribution over states, weighting their importance

The parameterized value function approximation

A common choice: The distribution over states induced by the policy $\pi$, with the intuition that we care about more heavily weighting the error for states that we frequently visit under the current policy.

Optimize using (stochastic) gradient descent.

# Learning the value function from on-policy data

We want to minimize $R(\theta)$,

$$R(\theta) = \mathbb{E}_{s\sim\mu}\left[\left(V^{\pi}(s) - V_{\theta}(s)\right)^2\right] = \sum_{s\in S}\mu(s)\left(V^{\pi}(s) - V_{\theta}(s)\right)^2$$

But, we can't evaluate $\mu(s)$ in practice.

Instead, we can sample the distribution over states $s_i$ by running $\pi$ in the environment and use this dataset to define the empirical risk:

$$\hat{R}(\theta) = \frac{1}{N}\sum_{i=1}^{N}\left(V^{\pi}(s_i) - V_{\theta}(s_i)\right)^2$$

# Learning the value function from on-policy data

We want to minimize $R(\theta)$,

$$R(\theta) = \mathbb{E}_{s\sim\mu}\left[\left(V^{\pi}(s) - V_{\theta}(s)\right)^2\right] = \sum_{s\in S}\mu(s)\left(V^{\pi}(s) - V_{\theta}(s)\right)^2$$

But, we can't evaluate $\mu(s)$ in practice.

Instead, we can sample the distribution over states $s_i$ by running $\pi$ in the environment and use this dataset to define the empirical risk:

$$\hat{R}(\theta) = \frac{1}{N}\sum_{i=1}^{N}\left(V^{\pi}(s_i) - V_{\theta}(s_i)\right)^2$$

Optimize using (stochastic) gradient descent.

# (Stochastic) gradient descent for learning the value function from on-policy data

For gradient descent, we need to compute $\nabla_\theta \hat{R}(\theta)$.

$$\nabla_\theta \hat{R}(\theta) = \nabla_\theta \left\{ \frac{1}{N} \sum_{i=1}^{N} \left[ V^\pi(s_i) - V_\theta(s_i) \right]^2 \right\}$$

$$= \frac{1}{N} \sum_{i=1}^{N} 2 \left[ V^\pi(s_i) - V_\theta(s_i) \right] * \nabla_\theta V_\theta(s_i)$$

Update rule

$$\implies \theta_{t+1} = \theta_t + \alpha \sum_{i=1}^{N} \left( V^\pi(s_i) - V_\theta(s_i) \right) \nabla_\theta V_\theta(s_i)$$

# (Stochastic) gradient descent for learning the value function from on-policy data

For gradient descent, we need to compute $\nabla_\theta \hat{R}(\theta)$.

$$\nabla_\theta \hat{R}(\theta) = \nabla_\theta \left\{ \frac{1}{N} \sum_{i=1}^{N} \left[ V^\pi(s_i) - V_\theta(s_i) \right]^2 \right\}$$
$$= \frac{1}{N} \sum_{i=1}^{N} 2 \left[ V^\pi(s_i) - V_\theta(s_i) \right] * \nabla_\theta V_\theta(s_i)$$

Update rule

$$\implies \theta_{t+1} = \theta_t + \alpha \sum_{i=1}^{N} \left( V^\pi(s_i) - V_\theta(s_i) \right) \nabla_\theta V_\theta(s_i)$$

We want to learn online while executing the policy in the environment.

# (Stochastic) gradient descent for learning the value function from on-policy data

For gradient descent, we need to compute $\nabla_\theta \hat{R}(\theta)$.

$$\nabla_\theta \hat{R}(\theta) = \nabla_\theta \left\{ \frac{1}{N} \sum_{i=1}^{N} \left[ V^\pi(s_i) - V_\theta(s_i) \right]^2 \right\}$$
$$= \frac{1}{N} \sum_{i=1}^{N} 2 \left[ V^\pi(s_i) - V_\theta(s_i) \right] * \nabla_\theta V_\theta(s_i)$$

Update rule

$$\implies \theta_{t+1} = \theta_t + \alpha \sum_{i=1}^{N} \left( V^\pi(s_i) - V_\theta(s_i) \right) \nabla_\theta V_\theta(s_i)$$

We want to learn online while executing the policy in the environment.

Stochastic gradient descent: use on-policy samples $S_t \sim \mu$ instead of computing the full expectation defining $\nabla_\theta \hat{R}(\theta)$.

$$\theta_{t+1} = \theta_t + \alpha \left( V^\pi(S_t) - V_\theta(S_t) \right) \nabla_\theta V_\theta(S_t)$$

That is, after each timestep $t$ executing $\pi$ in the environment, use the observed state $S_t$ to take a gradient step.

# (Stochastic) gradient descent for learning the value function from on-policy data

For gradient descent, we need to compute $\nabla_\theta \hat{R}(\theta)$.

$$\nabla_\theta \hat{R}(\theta) = \nabla_\theta \left\{ \frac{1}{N} \sum_{i=1}^{N} \left[ V^\pi(s_i) - V_\theta(s_i) \right]^2 \right\}$$
$$= \frac{1}{N} \sum_{i=1}^{N} 2 \left[ V^\pi(s_i) - V_\theta(s_i) \right] * \nabla_\theta V_\theta(s_i)$$

Update rule

$$\implies \theta_{t+1} = \theta_t + \alpha \sum_{i=1}^{N} \left( V^\pi(s_i) - V_\theta(s_i) \right) \nabla_\theta V_\theta(s_i)$$

We want to learn online while executing the policy in the environment.

Stochastic gradient descent: use on-policy samples $S_t \sim \mu$ instead of computing the full expectation defining $\nabla_\theta \hat{R}(\theta)$.

$$\theta_{t+1} = \theta_t + \alpha \left( V^\pi(S_t) - V_\theta(S_t) \right) \nabla_\theta V_\theta(S_t)$$

That is, after each timestep $t$ executing $\pi$ in the environment, use the observed state $S_t$ to take a gradient step.

Question: How do we get the value of $V^\pi(s)$ used to define the update rule??

$$\theta_{t+1} = \theta_t + \alpha \left( V^\pi(S_t) - V_\theta(S_t) \right) \nabla_\theta V_\theta(S_t)$$

# (Stochastic) gradient descent for learning the value function from on-policy data

Question: How do we get the value of $V^{\pi}(s)$ used to define the update rule??

$$\theta_{t+1} = \theta_t + \alpha \left( V^{\pi}(S_t) - V_{\theta}(S_t) \right) \nabla_{\theta} V_{\theta}(S_t)$$

# (Stochastic) gradient descent for learning the value function from on-policy data

Question: How do we get the value of $V^{\pi}(s)$ used to define the update rule??

$$\theta_{t+1} = \theta_t + \alpha \left( V^{\pi}(S_t) - V_{\theta}(S_t) \right) \nabla_{\theta} V_{\theta}(S_t)$$

Instead, use $U_t$, a stochastic approximation of $V^{\pi}(S_t)$ to define the gradient.

$$\theta_{t+1} = \theta_t + \alpha \left( U_t - V_{\theta}(S_t) \right) \nabla_{\theta} V_{\theta}(S_t)$$

# (Stochastic) gradient descent for learning the value function from on-policy data

Question: How do we get the value of $V^{\pi}(s)$ used to define the update rule??

$$\theta_{t+1} = \theta_t + \alpha \left( V^{\pi}(S_t) - V_\theta(S_t) \right) \nabla_\theta V_\theta(S_t)$$

Instead, use $U_t$, a stochastic approximation of $V^{\pi}(S_t)$ to define the gradient.

$$\theta_{t+1} = \theta_t + \alpha \left( U_t - V_\theta(S_t) \right) \nabla_\theta V_\theta(S_t)$$

There are different choices for $U_t$, for example, we could use Monte Carlo estimates of return $G_t$.

$$\theta_{t+1} = \theta_t + \alpha \left( V^{\pi}(S_t) - V_\theta(S_t) \right) \nabla_\theta V_\theta(S_t) \longrightarrow \theta_{t+1} = \theta_t + \alpha \left( U_t - V_\theta(S_t) \right) \nabla_\theta V_\theta(S_t) \longrightarrow \theta_{t+1} = \theta_t + \alpha \left( G_t - V_\theta(S_t) \right) \nabla_\theta V_\theta(S_t)$$

# (Stochastic) gradient descent for learning the value function from on-policy data

Question: How do we get the value of $V^\pi(s)$ used to define the update rule??

$$\theta_{t+1} = \theta_t + \alpha \left( V^\pi(S_t) - V_\theta(S_t) \right) \nabla_\theta V_\theta(S_t)$$

Instead, use $U_t$, a stochastic approximation of $V^\pi(S_t)$ to define the gradient.

$$\theta_{t+1} = \theta_t + \alpha \left( U_t - V_\theta(S_t) \right) \nabla_\theta V_\theta(S_t)$$

There are different choices for $U_t$, for example, we could use Monte Carlo estimates of return $G_t$.

$$\theta_{t+1} = \theta_t + \alpha \left( V^\pi(S_t) - V_\theta(S_t) \right) \nabla_\theta V_\theta(S_t) \longrightarrow \theta_{t+1} = \theta_t + \alpha \left( U_t - V_\theta(S_t) \right) \nabla_\theta V_\theta(S_t) \longrightarrow \theta_{t+1} = \theta_t + \alpha \left( G_t - V_\theta(S_t) \right) \nabla_\theta V_\theta(S_t)$$

Recall: $V^\pi(s) = \mathbb{E}[G_t \mid S_t = s, \pi]$

Sample episodes $\tau^{(i)} = s_0^{(i)} a_0^{(i)} r_0^{(i)} s_1^{(i)} a_1^{(i)} r_1^{(i)} \ldots s_{T^{(i)}}^{(i)}$.

For every $t$, compute

$$g_t^{(i)} = r_{t+1}^{(i)} + \gamma r_{t+2}^{(i)} + \ldots + \gamma^{T^{(i)}-t-1} r_{T^{(i)}-1}.$$

Estimate

$$\hat{V}^\pi(s) = \frac{1}{N(s)} \sum_{i=1}^{N} \sum_{t=0}^{T} \mathbb{I}\{S_t = s\} g_t^{(i)}$$

# (Stochastic) gradient descent for learning the value function from on-policy data

Question: How do we get the value of $V^\pi(s)$ used to define the update rule??

$$\theta_{t+1} = \theta_t + \alpha \left( V^\pi(S_t) - V_\theta(S_t) \right) \nabla_\theta V_\theta(S_t)$$

Instead, use $U_t$, a stochastic approximation of $V^\pi(S_t)$ to define the gradient.

$$\theta_{t+1} = \theta_t + \alpha \left( U_t - V_\theta(S_t) \right) \nabla_\theta V_\theta(S_t)$$

There are different choices for $U_t$, for example, we could use Monte Carlo estimates of return $G_t$.

$$\theta_{t+1} = \theta_t + \alpha \left( V^\pi(S_t) - V_\theta(S_t) \right) \nabla_\theta V_\theta(S_t) \longrightarrow \theta_{t+1} = \theta_t + \alpha \left( U_t - V_\theta(S_t) \right) \nabla_\theta V_\theta(S_t) \longrightarrow \theta_{t+1} = \theta_t + \alpha \left( G_t - V_\theta(S_t) \right) \nabla_\theta V_\theta(S_t)$$

Recall: $V^\pi(s) = \mathbb{E}[G_t \mid S_t = s, \pi]$

Sample episodes $\tau^{(i)} = s_0^{(i)} a_0^{(i)} r_0^{(i)} s_1^{(i)} a_1^{(i)} r_1^{(i)} \ldots s_{T^{(i)}}^{(i)}$.

For every $t$, compute

$$g_t^{(i)} = r_{t+1}^{(i)} + \gamma r_{t+2}^{(i)} + \ldots + \gamma^{T^{(i)}-t-1} r_{T^{(i)}-1}.$$

Estimate

$$\hat{V}^\pi(s) = \frac{1}{N(s)} \sum_{i=1}^{N} \sum_{t=0}^{T} \mathbb{I}\{S_t = s\} g_t^{(i)}$$

**Gradient Monte Carlo Algorithm for Estimating $\hat{v} \approx v_\pi$**

Input: the policy $\pi$ to be evaluated
Input: a differentiable function $\hat{v} : \mathcal{S} \times \mathbb{R}^d \to \mathbb{R}$
Algorithm parameter: step size $\alpha > 0$
Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Loop forever (for each episode):
  Generate an episode $S_0, A_0, R_1, S_1, A_1, \ldots, R_T, S_T$ using $\pi$
  Loop for each step of episode, $t = 0, 1, \ldots, T-1$:
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha \big[ G_t - \hat{v}(S_t, \mathbf{w}) \big] \nabla \hat{v}(S_t, \mathbf{w})$

Figure courtesy of Sutton & Barto (2018)

# (Stochastic) gradient descent for learning the value function from on-policy data

**Gradient Monte Carlo Algorithm for Estimating $\hat{v} \approx v_\pi$**

Input: the policy $\pi$ to be evaluated
Input: a differentiable function $\hat{v} : \mathcal{S} \times \mathbb{R}^d \to \mathbb{R}$
Algorithm parameter: step size $\alpha > 0$
Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Loop forever (for each episode):
  Generate an episode $S_0, A_0, R_1, S_1, A_1, \ldots, R_T, S_T$ using $\pi$
  Loop for each step of episode, $t = 0, 1, \ldots, T-1$:
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha\big[G_t - \hat{v}(S_t, \mathbf{w})\big]\nabla \hat{v}(S_t, \mathbf{w})$

Figure courtesy of Sutton & Barto (2018)

Since $G_t$ is an unbiased estimate of $V^\pi(S_t)$, this method will converge to a local optimum $V_\theta(s)$ approximation of $V^\pi(s)$.

Assuming the usual stochastic approximation conditions for decreasing $\alpha$.

$$\sum_{t=1}^{\infty} \alpha_t = \infty \quad \text{and} \quad \sum_{t=1}^{\infty} \alpha_t^2 < \infty$$

Note: This requires entire episodes be rolled out before we can compute a single gradient update.

- Instead, replace $G_t$ with TD(0) estimator $R_t + \gamma V_\theta(S_{t+1})$ of $V^\pi(S_{t+1})$?

# **Semi-gradient** methods for learning the value function from on-policy data

Idea: Replace $G_t$ with $R_t + \gamma V_\theta(S_{t+1})$ as the estimate $U_t$ of $V^\pi(S_t)$ in the SGD update.

$$\theta_{t+1} = \theta_t + \alpha \left(V^\pi(S_t) - V_\theta(S_t)\right) \nabla_\theta V_\theta(S_t) \longrightarrow \theta_{t+1} = \theta_t + \alpha \left(U_t - V_\theta(S_t)\right) \nabla_\theta V_\theta(S_t) \longrightarrow \boxed{\theta_{t+1} = \theta_t + \alpha \left(R_t + \gamma V_\theta(S_{t+1}) - V_\theta(S_t)\right) \nabla_\theta V_\theta(S_t)}$$

# **Semi-gradient** methods for learning the value function from on-policy data

Idea: Replace $G_t$ with $R_t + \gamma V_\theta(S_{t+1})$ as the estimate $U_t$ of $V^\pi(S_t)$ in the SGD update.

$$\theta_{t+1} = \theta_t + \alpha \left(V^\pi(S_t) - V_\theta(S_t)\right) \nabla_\theta V_\theta(S_t) \longrightarrow \theta_{t+1} = \theta_t + \alpha \left(U_t - V_\theta(S_t)\right) \nabla_\theta V_\theta(S_t) \longrightarrow \boxed{\theta_{t+1} = \theta_t + \alpha \left(R_t + \gamma V_\theta(S_{t+1}) - V_\theta(S_t)\right) \nabla_\theta V_\theta(S_t)}$$

**Semi-gradient TD(0) for estimating $\hat{v} \approx v_\pi$**

Input: the policy $\pi$ to be evaluated
Input: a differentiable function $\hat{v} : \mathcal{S}^+ \times \mathbb{R}^d \to \mathbb{R}$ such that $\hat{v}(\text{terminal},\cdot) = 0$
Algorithm parameter: step size $\alpha > 0$
Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Loop for each episode:
  Initialize $S$
  Loop for each step of episode:
    Choose $A \sim \pi(\cdot|S)$
    Take action $A$, observe $R, S'$
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha\big[R + \gamma\hat{v}(S',\mathbf{w}) - \hat{v}(S,\mathbf{w})\big]\nabla\hat{v}(S,\mathbf{w})$
    $S \leftarrow S'$
  until $S$ is terminal

Figure courtesy of Sutton & Barto (2018)

# **Semi-gradient** methods for learning the value function from on-policy data

Idea: Replace $G_t$ with $R_t + \gamma V_\theta(S_{t+1})$ as the estimate $U_t$ of $V^\pi(S_t)$ in the SGD update.

$$\theta_{t+1} = \theta_t + \alpha \left(V^\pi(S_t) - V_\theta(S_t)\right) \nabla_\theta V_\theta(S_t) \longrightarrow \theta_{t+1} = \theta_t + \alpha \left(U_t - V_\theta(S_t)\right) \nabla_\theta V_\theta(S_t) \longrightarrow \boxed{\theta_{t+1} = \theta_t + \alpha \left(R_t + \gamma V_\theta(S_{t+1}) - V_\theta(S_t)\right) \nabla_\theta V_\theta(S_t)}$$

**Semi-gradient TD(0) for estimating $\hat{v} \approx v_\pi$**

Input: the policy $\pi$ to be evaluated
Input: a differentiable function $\hat{v} : \mathcal{S}^+ \times \mathbb{R}^d \to \mathbb{R}$ such that $\hat{v}(\text{terminal}, \cdot) = 0$
Algorithm parameter: step size $\alpha > 0$
Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Loop for each episode:
  Initialize $S$
  Loop for each step of episode:
    Choose $A \sim \pi(\cdot|S)$
    Take action $A$, observe $R, S'$
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha\left[R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})\right] \nabla \hat{v}(S, \mathbf{w})$
    $S \leftarrow S'$
  until $S$ is terminal

Figure courtesy of Sutton & Barto (2018)

Note, this is an instance of *bootstrapping*: we are using our current estimate $V_\theta(S_{t+1})$ to define part of the learning target!

# **Semi-gradient** methods for learning the value function from on-policy data

Idea: Replace $G_t$ with $R_t + \gamma V_\theta(S_{t+1})$ as the estimate $U_t$ of $V^\pi(S_t)$ in the SGD update.

$$\theta_{t+1} = \theta_t + \alpha \left(V^\pi(S_t) - V_\theta(S_t)\right) \nabla_\theta V_\theta(S_t) \longrightarrow \theta_{t+1} = \theta_t + \alpha \left(U_t - V_\theta(S_t)\right) \nabla_\theta V_\theta(S_t) \longrightarrow \boxed{\theta_{t+1} = \theta_t + \alpha \left(R_t + \gamma V_\theta(S_{t+1}) - V_\theta(S_t)\right) \nabla_\theta V_\theta(S_t)}$$

**Semi-gradient TD(0) for estimating $\hat{v} \approx v_\pi$**

Input: the policy $\pi$ to be evaluated
Input: a differentiable function $\hat{v} : \mathcal{S}^+ \times \mathbb{R}^d \to \mathbb{R}$ such that $\hat{v}(\text{terminal}, \cdot) = 0$
Algorithm parameter: step size $\alpha > 0$
Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Loop for each episode:
  Initialize $S$
  Loop for each step of episode:
    Choose $A \sim \pi(\cdot|S)$
    Take action $A$, observe $R, S'$
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha\left[R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})\right] \nabla \hat{v}(S, \mathbf{w})$
    $S \leftarrow S'$
  until $S$ is terminal

Figure courtesy of Sutton & Barto (2018)

Note, this is an instance of *bootstrapping*: we are using our current estimate $V_\theta(S_{t+1})$ to define part of the learning target!

Also, our update rule no longer corresponds to a proper gradient step for our previously-defined loss function

$$\theta_{t+1} = \theta_t + \alpha \left(R_t + \gamma V_\theta(S_{t+1}) - V_\theta(S_t)\right) \nabla_\theta V_\theta(S_t)$$

# **Semi-gradient** methods for learning the value function from on-policy data

Idea: Replace $G_t$ with $R_t + \gamma V_\theta(S_{t+1})$ as the estimate $U_t$ of $V^\pi(S_t)$ in the SGD update.

$$\theta_{t+1} = \theta_t + \alpha \left(V^\pi(S_t) - V_\theta(S_t)\right) \nabla_\theta V_\theta(S_t) \longrightarrow \theta_{t+1} = \theta_t + \alpha \left(U_t - V_\theta(S_t)\right) \nabla_\theta V_\theta(S_t) \longrightarrow \boxed{\theta_{t+1} = \theta_t + \alpha \left(R_t + \gamma V_\theta(S_{t+1}) - V_\theta(S_t)\right) \nabla_\theta V_\theta(S_t)}$$

**Semi-gradient TD(0) for estimating $\hat{v} \approx v_\pi$**

Input: the policy $\pi$ to be evaluated
Input: a differentiable function $\hat{v} : \mathcal{S}^+ \times \mathbb{R}^d \to \mathbb{R}$ such that $\hat{v}(\text{terminal}, \cdot) = 0$
Algorithm parameter: step size $\alpha > 0$
Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Loop for each episode:
  Initialize $S$
  Loop for each step of episode:
    Choose $A \sim \pi(\cdot|S)$
    Take action $A$, observe $R, S'$
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha\left[R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})\right] \nabla \hat{v}(S, \mathbf{w})$
    $S \leftarrow S'$
  until $S$ is terminal

Figure courtesy of Sutton & Barto (2018)

Note, this is an instance of *bootstrapping*: we are using our current estimate $V_\theta(S_{t+1})$ to define part of the learning target!

Also, our update rule no longer corresponds to a proper gradient step for our previously-defined loss function

$$\theta_{t+1} = \theta_t + \alpha \left(R_t + \gamma V_\theta(S_{t+1}) - V_\theta(S_t)\right) \nabla_\theta V_\theta(S_t)$$

Instead, we compute a so-called ***semi-gradient:*** we effectively *ignore* the presence of $V_\theta(S_{t+1})$ in the target, and only consider the contribution of the estimate $V_\theta(S_t)$ to the gradient.

# **Semi-gradient** methods for learning the value function from on-policy data

Idea: Replace $G_t$ with $R_t + \gamma V_\theta(S_{t+1})$ as the estimate $U_t$ of $V^\pi(S_t)$ in the SGD update.

$$\theta_{t+1} = \theta_t + \alpha \left(V^\pi(S_t) - V_\theta(S_t)\right) \nabla_\theta V_\theta(S_t) \longrightarrow \theta_{t+1} = \theta_t + \alpha \left(U_t - V_\theta(S_t)\right) \nabla_\theta V_\theta(S_t) \longrightarrow \boxed{\theta_{t+1} = \theta_t + \alpha \left(R_t + \gamma V_\theta(S_{t+1}) - V_\theta(S_t)\right) \nabla_\theta V_\theta(S_t)}$$

**Semi-gradient TD(0) for estimating $\hat{v} \approx v_\pi$**

Input: the policy $\pi$ to be evaluated
Input: a differentiable function $\hat{v} : \mathcal{S}^+ \times \mathbb{R}^d \to \mathbb{R}$ such that $\hat{v}(\text{terminal}, \cdot) = 0$
Algorithm parameter: step size $\alpha > 0$
Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Loop for each episode:
  Initialize $S$
  Loop for each step of episode:
    Choose $A \sim \pi(\cdot|S)$
    Take action $A$, observe $R, S'$
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha\left[R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})\right] \nabla \hat{v}(S, \mathbf{w})$
    $S \leftarrow S'$
  until $S$ is terminal

Figure courtesy of Sutton & Barto (2018)

Note, this is an instance of *bootstrapping*: we are using our current estimate $V_\theta(S_{t+1})$ to define part of the learning target!

Also, our update rule no longer corresponds to a proper gradient step for our previously-defined loss function

$$\theta_{t+1} = \theta_t + \alpha \left(R_t + \gamma V_\theta(S_{t+1}) - V_\theta(S_t)\right) \nabla_\theta V_\theta(S_t)$$

Instead, we compute a so-called ***semi-gradient:*** we effectively *ignore* the presence of $V_\theta(S_{t+1})$ in the target, and only consider the contribution of the estimate $V_\theta(S_t)$ to the gradient.

Semi-gradient methods are biased relative to minimizing the mean-squared Bellman error.
However, in practice they are far more efficient and usually more stable than full residual-gradient methods.

# **Semi-gradient** methods for learning the value function from on-policy data

Idea: Replace $G_t$ with $R_t + \gamma V_\theta(S_{t+1})$ as the estimate $U_t$ of $V^\pi(S_t)$ in the SGD update.

$$\theta_{t+1} = \theta_t + \alpha \left(V^\pi(S_t) - V_\theta(S_t)\right) \nabla_\theta V_\theta(S_t) \longrightarrow \theta_{t+1} = \theta_t + \alpha \left(U_t - V_\theta(S_t)\right) \nabla_\theta V_\theta(S_t) \longrightarrow \boxed{\theta_{t+1} = \theta_t + \alpha \left(R_t + \gamma V_\theta(S_{t+1}) - V_\theta(S_t)\right) \nabla_\theta V_\theta(S_t)}$$

# **Semi-gradient** methods for learning the value function from on-policy data

Idea: Replace $G_t$ with $R_t + \gamma V_\theta(S_{t+1})$ as the estimate $U_t$ of $V^\pi(S_t)$ in the SGD update.

$$\theta_{t+1} = \theta_t + \alpha \left(V^\pi(S_t) - V_\theta(S_t)\right) \nabla_\theta V_\theta(S_t) \longrightarrow \theta_{t+1} = \theta_t + \alpha \left(U_t - V_\theta(S_t)\right) \nabla_\theta V_\theta(S_t) \longrightarrow \boxed{\theta_{t+1} = \theta_t + \alpha \left(R_t + \gamma V_\theta(S_{t+1}) - V_\theta(S_t)\right) \nabla_\theta V_\theta(S_t)}$$

Taking the full gradient of mean-squared Bellman error:

Consider the target $U_t = R_t + \gamma V_\theta(s_{t+1})$ and corresponding risk: $R(\theta) = \mathbb{E}_{(s_t, a_t, r_t, s_{t+1}) \sim \pi} \left[ \left( r_t + \gamma V_\theta(s_{t+1}) - V_\theta(s_t) \right)^2 \right]$

The loss function $\mathscr{L}(\theta) = \left( r_t + \gamma V_\theta(s_{t+1}) - V_\theta(s_t) \right)^2$ has the gradient: $\nabla_\theta \mathscr{L}(\theta) = \left( r_t + \gamma V_\theta(s_{t+1}) - V_\theta(s_t) \right) \cdot \left( \gamma \nabla_\theta V_\theta(s_{t+1}) - \nabla_\theta V_\theta(s_t) \right)$

Which would yield the SGD update rule: $\theta_{t+1} = \theta_t + \alpha \left( R_t + \gamma V_\theta(S_{t+1}) - V_\theta(S_t) \right) \cdot \left( \nabla_\theta V_\theta(S_t) - \gamma \nabla_\theta V_\theta(S_{t+1}) \right)$

# **Semi-gradient** methods for learning the value function from on-policy data

Idea: Replace $G_t$ with $R_t + \gamma V_\theta(S_{t+1})$ as the estimate $U_t$ of $V^\pi(S_t)$ in the SGD update.

$$\theta_{t+1} = \theta_t + \alpha \left(V^\pi(S_t) - V_\theta(S_t)\right) \nabla_\theta V_\theta(S_t) \longrightarrow \theta_{t+1} = \theta_t + \alpha \left(U_t - V_\theta(S_t)\right) \nabla_\theta V_\theta(S_t) \longrightarrow \boxed{\theta_{t+1} = \theta_t + \alpha \left(R_t + \gamma V_\theta(S_{t+1}) - V_\theta(S_t)\right) \nabla_\theta V_\theta(S_t)}$$

Taking the full gradient of mean-squared Bellman error:

Consider the target $U_t = R_t + \gamma V_\theta(s_{t+1})$ and corresponding risk: $R(\theta) = \mathbb{E}_{(s_t, a_t, r_t, s_{t+1}) \sim \pi} \left[ \left( r_t + \gamma V_\theta(s_{t+1}) - V_\theta(s_t) \right)^2 \right]$

The loss function $\mathscr{L}(\theta) = \left( r_t + \gamma V_\theta(s_{t+1}) - V_\theta(s_t) \right)^2$ has the gradient: $\nabla_\theta \mathscr{L}(\theta) = \left( r_t + \gamma V_\theta(s_{t+1}) - V_\theta(s_t) \right) \cdot \left( \gamma \nabla_\theta V_\theta(s_{t+1}) - \nabla_\theta V_\theta(s_t) \right)$

Which would yield the SGD update rule: $\theta_{t+1} = \theta_t + \alpha \left( R_t + \gamma V_\theta(S_{t+1}) - V_\theta(S_t) \right) \cdot \left( \nabla_\theta V_\theta(S_t) - \gamma \nabla_\theta V_\theta(S_{t+1}) \right)$

Taking a “semi-gradient” of the mean-squared Bellman error:

Instead, we compute a so-called ***semi-gradient:*** we effectively *ignore* the presence of $V_\theta(S_{t+1})$ in the target, and only consider the contribution of the estimate $V_\theta(S_t)$ to the gradient.

$$\text{semi-grad}(\mathscr{L}(\theta)) = \left( r_t + \gamma V_\theta(s_{t+1}) - V_\theta(s_t) \right) \nabla_\theta V_\theta(s_t) \longrightarrow \theta_{t+1} = \theta_t + \alpha \left( R_t + \gamma V_\theta(S_{t+1}) - V_\theta(S_t) \right) \nabla_\theta V_\theta(S_t)$$

# What are the benefits of semi-gradient methods?

- Benefits: we don't need the full rollout every step.
  ⇒ typically have faster learning in comparison to $U_t = G_t$
- Enables online learning / continual learning algorithms.
- In on-policy case with linear function approximation, this semi-gradient method converges to a unique fixed point.
  ↳ that fixed point is <u>not</u> the fixed point of the Bellman error, but rather the fixed point of a "projected Bellman operator".

# Linear function approximation for value prediction

$\sum_{i=1}^{d} \theta_i \phi_i(s)$

An important setting to consider: $V_\theta(s) = \theta^T \phi(s)$ is a linear combination of state features $\phi(s) \in \mathbb{R}^d$.

# Linear function approximation for value prediction

An important setting to consider: $V_\theta(s) = \theta^T \phi(s)$ is a linear combination of state features $\phi(s) \in \mathbb{R}^d$.

Feature vectors could encode, for example:

- A one-hot encoding $\phi(s) = (0,\ldots,0,1,0,\ldots,0)$ in the tabular setting.
- Hand-crafted features (e.g., in a gridworld, $\phi(s) = (x, y)$ the location of the agent).
- Radial basis functions $\phi_j(s) = \exp\left(\frac{-||s - c_j||^2}{2\sigma^2}\right)$
- Learned embeddings: imaging $\phi(s)$ coming from the output of a neural network, and $\theta^T \phi(s)$ corresponding to the final linear layer of the network.

# Linear function approximation for value prediction

An important setting to consider: $V_\theta(s) = \theta^T \phi(s)$ is a linear combination of state features $\phi(s) \in \mathbb{R}^d$.

Feature vectors could encode, for example:

- A one-hot encoding $\phi(s) = (0,\ldots,0,1,0,\ldots,0)$ in the tabular setting.
- Hand-crafted features (e.g., in a gridworld, $\phi(s) = (x, y)$ the location of the agent).
- Radial basis functions $\phi_j(s) = \exp\left(-\frac{||s - c_j||^2}{2\sigma^2}\right)$
- Learned embeddings: imaging $\phi(s)$ coming from the output of a neural network, and $\theta^T \phi(s)$ corresponding to the final linear layer of the network.

This setting is simple enough to do some analysis, while also highlighting some of the fundamental limitations of RL with function approximation.

# Linear function approximation for value prediction

An important setting to consider: $V_\theta(s) = \theta^T \phi(s)$ is a linear combination of state features $\phi(s) \in \mathbb{R}^d$.

Feature vectors could encode, for example:

- A one-hot encoding $\phi(s) = (0,\ldots,0,1,0,\ldots,0)$ in the tabular setting.
- Hand-crafted features (e.g., in a gridworld, $\phi(s) = (x, y)$ the location of the agent).
- Radial basis functions $\phi_j(s) = \exp\left(-\frac{||s - c_j||^2}{2\sigma^2}\right)$
- Learned embeddings: imaging $\phi(s)$ coming from the output of a neural network, and $\theta^T \phi(s)$ corresponding to the final linear layer of the network.

This setting is simple enough to do some analysis, while also highlighting some of the fundamental limitations of RL with function approximation.

When $V_\theta(s) = \theta^T \phi(s)$, the semi-gradient TD(0) update becomes:

$$\begin{aligned}\theta_{t+1} &= \theta_t + \alpha \left[R_t + \gamma V_\theta(S_{t+1}) - V_\theta(S_t)\right] \phi(S_t) \\ &= \theta_t + \alpha \left[R_t \phi(S_t) - \phi(S_t)\left(\phi(S_t) - \gamma \phi(S_{t+1})\right)^T \theta\right]\end{aligned}$$

# Linear function approximation for value prediction

An important setting to consider: $V_\theta(s) = \theta^T \phi(s)$ is a linear combination of state features $\phi(s) \in \mathbb{R}^d$.

Feature vectors could encode, for example:

- A one-hot encoding $\phi(s) = (0,\ldots,0,1,0,\ldots,0)$ in the tabular setting.
- Hand-crafted features (e.g., in a gridworld, $\phi(s) = (x, y)$ the location of the agent).
- Radial basis functions $\phi_j(s) = \exp\left(-\frac{||s - c_j||^2}{2\sigma^2}\right)$
- Learned embeddings: imaging $\phi(s)$ coming from the output of a neural network, and $\theta^T \phi(s)$ corresponding to the final linear layer of the network.

This setting is simple enough to do some analysis, while also highlighting some of the fundamental limitations of RL with function approximation.

When $V_\theta(s) = \theta^T \phi(s)$, the semi-gradient TD(0) update becomes:

$$\begin{aligned}\theta_{t+1} &= \theta_t + \alpha \left[R_t + \gamma V_\theta(S_{t+1}) - V_\theta(S_t)\right] \phi(S_t) \\ &= \theta_t + \alpha \left[R_t \phi(S_t) - \phi(S_t)\left(\phi(S_t) - \gamma \phi(S_{t+1})\right)^T \theta\right]\end{aligned}$$

Define $A = \mathbb{E}\left[\phi(S_t)\left(\phi(S_t) - \gamma\phi(S_{t+1})\right)^T\right] \in \mathbb{R}^{d\times d}$ $b = \mathbb{E}\left[R_t \phi(S_t)\right] \in \mathbb{R}^d$

Then, on-policy semi-gradient TD(0) converges to the weight vector $\theta_{TD} = A^{-1} b$. Proof: Sutton & Barto (2018), pg. 206.

# Linear function approximation for value prediction

An important setting to consider: $V_\theta(s) = \theta^T \phi(s)$ is a linear combination of state features $\phi(s) \in \mathbb{R}^d$.

Feature vectors could encode, for example:

- A one-hot encoding $\phi(s) = (0,\ldots,0,1,0,\ldots,0)$ in the tabular setting.
- Hand-crafted features (e.g., in a gridworld, $\phi(s) = (x, y)$ the location of the agent).
- Radial basis functions $\phi_j(s) = \exp\left(-\frac{||s - c_j||^2}{2\sigma^2}\right)$
- Learned embeddings: imaging $\phi(s)$ coming from the output of a neural network, and $\theta^T \phi(s)$ corresponding to the final linear layer of the network.

This setting is simple enough to do some analysis, while also highlighting some of the fundamental limitations of RL with function approximation.

When $V_\theta(s) = \theta^T \phi(s)$, the semi-gradient TD(0) update becomes:

$$\begin{aligned}\theta_{t+1} &= \theta_t + \alpha \left[R_t + \gamma V_\theta(S_{t+1}) - V_\theta(S_t)\right] \phi(S_t) \\ &= \theta_t + \alpha \left[R_t \phi(S_t) - \phi(S_t)\left(\phi(S_t) - \gamma \phi(S_{t+1})\right)^T \theta\right]\end{aligned}$$

Define $A = \mathbb{E}\left[\phi(S_t)\left(\phi(S_t) - \gamma\phi(S_{t+1})\right)^T\right] \in \mathbb{R}^{d\times d}$ $b = \mathbb{E}\left[R_t \phi(S_t)\right] \in \mathbb{R}^d$

Then, on-policy semi-gradient TD(0) converges to the weight vector $\theta_{TD} = A^{-1} b$. Proof: Sutton & Barto (2018), pg. 206.

This is only true for on-policy sampling. In the case of off-policy sampling, $\theta_t$ is not guaranteed to converge at all.

# Example: State aggregation as linear function approximation

Consider a 1000-state version of random walk

Transitions are:

- To one of the 100 states to the left of the current state, w.p. 0.5
- To one of the 100 states to the right of the current state, w.p. 0.5

If the current state is within 100 states of either MDP edge, the probability mass of the missing states goes into the probability of transitioning into the terminal state on that side.

# Example: State aggregation as linear function approximation

Consider a 1000-state version of random walk

Transitions are:

- To one of the 100 states to the left of the current state, w.p. 0.5
- To one of the 100 states to the right of the current state, w.p. 0.5

If the current state is within 100 states of either MDP edge, the probability mass of the missing states goes into the probability of transitioning into the terminal state on that side.

Value function is approximated over *aggregated states*. I.e., the 1000 states are partitioned into consecutive groups of 100.

# Example: State aggregation as linear function approximation

Consider a 1000-state version of random walk

Transitions are:

To one of the 100 states to the left of the current state, w.p. 0.5

To one of the 100 states to the right of the current state, w.p. 0.5

If the current state is within 100 states of either MDP edge, the probability mass of the missing states goes into the probability of transitioning into the terminal state on that side.

Value function is approximated over *aggregated states*. I.e., the 1000 states are partitioned into consecutive groups of 100.

We can formalize $V_\theta(s) = \theta^T \phi(s)$ by defining the feature vector $\phi(s) \in \mathbb{R}^d$ with $d = 10$ as:

$$\phi_j(s) = \begin{cases} 1 & \text{if } s \in \left[\frac{100}{d}(j-1)+1, \frac{1000}{j}\right] \\ 0 & \text{otherwise} \end{cases} \quad \text{For all} \quad j = 1,2,\ldots,10$$

# Example: State aggregation as linear function approximation

Consider a 1000-state version of random walk

Transitions are:

- To one of the 100 states to the left of the current state, w.p. 0.5
- To one of the 100 states to the right of the current state, w.p. 0.5

If the current state is within 100 states of either MDP edge, the probability mass of the missing states goes into the probability of transitioning into the terminal state on that side.

Value function is approximated over *aggregated states*. I.e., the 1000 states are partitioned into consecutive groups of 100.

We can formalize $V_\theta(s) = \theta^T \phi(s)$ by defining the feature vector $\phi(s) \in \mathbb{R}^d$ with $d = 10$ as:

$$\phi_j(s) = \begin{cases} 1 & \text{if } s \in \left[\frac{100}{d}(j-1)+1, \frac{1000}{j}\right] \\ 0 & \text{otherwise} \end{cases} \quad \text{For all} \quad j = 1,2,\ldots,10$$

$\Longrightarrow$ $\phi(s)$ is a d-dimensional binary vector, with exactly one entry equal to 1, corresponding to the aggregate state containing $s$.

$\theta_j$ then corresponds to the estimated value of aggregate state $j$.

# Example: State aggregation as linear function approximation

Consider a 1000-state version of random walk

Transitions are:

To one of the 100 states to the left of the current state, w.p. 0.5

To one of the 100 states to the right of the current state, w.p. 0.5

If the current state is within 100 states of either MDP edge, the probability mass of the missing states goes into the probability of transitioning into the terminal state on that side.

Value function is approximated over *aggregated states*. I.e., the 1000 states are partitioned into consecutive groups of 100.

We can formalize $V_\theta(s) = \theta^T \phi(s)$ by defining the feature vector $\phi(s) \in \mathbb{R}^d$ with $d = 10$ as:

$$\phi_j(s) = \begin{cases} 1 & \text{if } s \in \left[\frac{100}{d}(j-1)+1, \frac{1000}{j}\right] \\ 0 & \text{otherwise} \end{cases} \quad \text{For all} \quad j = 1,2,\ldots,10$$

$\Longrightarrow$ $\phi(s)$ is a d-dimensional binary vector, with exactly one entry equal to 1, corresponding to the aggregate state containing $s$.

$\theta_j$ then corresponds to the estimated value of aggregate state $j$.

**Gradient Monte Carlo Algorithm for Estimating $\hat{v} \approx v_\pi$**

Input: the policy $\pi$ to be evaluated
Input: a differentiable function $\hat{v} : \mathcal{S} \times \mathbb{R}^d \to \mathbb{R}$
Algorithm parameter: step size $\alpha > 0$
Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Loop forever (for each episode):
  Generate an episode $S_0, A_0, R_1, S_1, A_1, \ldots, R_T, S_T$ using $\pi$
  Loop for each step of episode, $t = 0, 1, \ldots, T-1$:
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha\left[G_t - \hat{v}(S_t, \mathbf{w})\right] \nabla \hat{v}(S_t, \mathbf{w})$

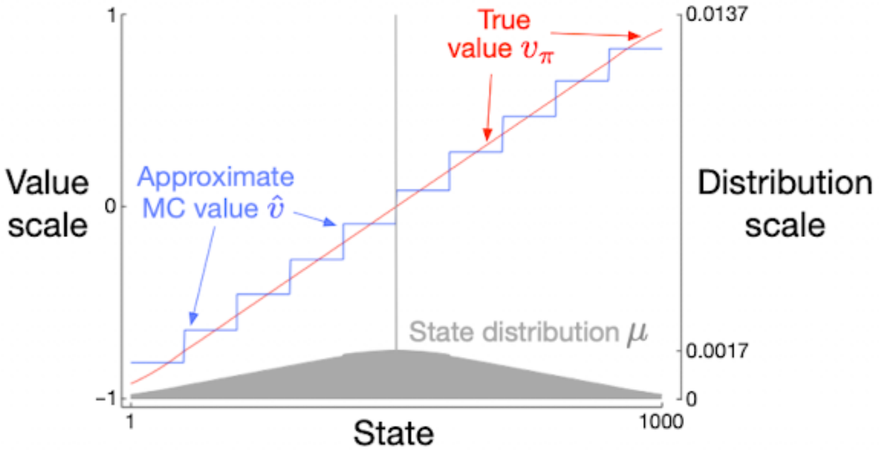



Figure courtesy of Sutton & Barto (2018)

# On-policy control via episodic semi-gradient value approximation

As in previous lectures, to do GPI/Control, we need $Q_\theta(s, a)$, a parameterized estimate of the state-action value function, instead of $V_\theta(s)$.

# On-policy control via episodic semi-gradient value approximation

As in previous lectures, to do GPI/Control, we need $Q_\theta(s, a)$, a parameterized estimate of the state-action value function, instead of $V_\theta(s)$.

This can be accomplished by defining the SGD update rule Monte Carlo estimates $G_t$ of returns, or using SARSA-style targets.

$$\theta_{t+1} = \theta_t + \alpha \left[U_t - Q_\theta(S_t, A_t)\right] \nabla_\theta Q_\theta(S_t, A_t)$$

Monte Carlo

$$\theta_{t+1} = \theta_t + \alpha \left[G_t - Q_\theta(S_t, A_t)\right] \nabla_\theta Q_\theta(S_t, A_t)$$

SARSA

$$\theta_{t+1} = \theta_t + \alpha \left[R_t + \gamma Q_\theta(S_{t+1}, A_{t+1}) - Q_\theta(S_t, A_t)\right] \nabla_\theta Q_\theta(S_t, A_t)$$

# On-policy control via episodic semi-gradient value approximation

As in previous lectures, to do GPI/Control, we need $Q_\theta(s, a)$, a parameterized estimate of the state-action value function, instead of $V_\theta(s)$.

This can be accomplished by defining the SGD update rule Monte Carlo estimates $G_t$ of returns, or using SARSA-style targets.

$$\theta_{t+1} = \theta_t + \alpha \left[U_t - Q_\theta(S_t, A_t)\right] \nabla_\theta Q_\theta(S_t, A_t)$$

Monte Carlo

$$\theta_{t+1} = \theta_t + \alpha \left[G_t - Q_\theta(S_t, A_t)\right] \nabla_\theta Q_\theta(S_t, A_t)$$

SARSA

$$\theta_{t+1} = \theta_t + \alpha \left[R_t + \gamma Q_\theta(S_{t+1}, A_{t+1}) - Q_\theta(S_t, A_t)\right] \nabla_\theta Q_\theta(S_t, A_t)$$

In the case of discrete action spaces, we can again use the $\epsilon$-greedy method to define the policy improvements in terms of the current estimate $Q_\theta(s, a)$

**Episodic Semi-gradient Sarsa for Estimating $\hat{q} \approx q_*$**

Input: a differentiable action-value function parameterization $\hat{q} : \mathcal{S} \times \mathcal{A} \times \mathbb{R}^d \to \mathbb{R}$
Algorithm parameters: step size $\alpha > 0$, small $\varepsilon > 0$
Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Loop for each episode:
  $S, A \leftarrow$ initial state and action of episode (e.g., $\varepsilon$-greedy)
  Loop for each step of episode:
    Take action $A$, observe $R, S'$
    If $S'$ is terminal:
      $\mathbf{w} \leftarrow \mathbf{w} + \alpha\big[R - \hat{q}(S, A, \mathbf{w})\big]\nabla\hat{q}(S, A, \mathbf{w})$
      Go to next episode
    Choose $A'$ as a function of $\hat{q}(S', \cdot, \mathbf{w})$ (e.g., $\varepsilon$-greedy)
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha\big[R + \gamma\hat{q}(S', A', \mathbf{w}) - \hat{q}(S, A, \mathbf{w})\big]\nabla\hat{q}(S, A, \mathbf{w})$
    $S \leftarrow S'$
    $A \leftarrow A'$

Figure courtesy of Sutton & Barto (2018)

# Mountain car example: On-policy control via episodic semi-gradient value approximation

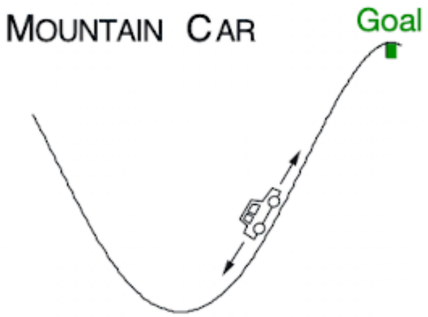



Figure courtesy of Sutton & Barto (2018)

Reward

$R(s) = -1$ for every state $s$, until the car passes the goal at the top right and so ends the episode.

Dynamics $x_{t+1} = \text{bound}(x_t + \dot{x}_{t+1})$

$\dot{x}_{t+1} = \text{bound}(\dot{x}_t + 0.001A_t - 0.0025cos(3x_t))$

Where bound( ) enforces $-1.2 \leq x \leq 0.5$ and $-0.07 \leq \dot{x} \leq 0.07$

Actions -1 (full throttle left), +1 (full throttle right), 0 (do nothing).

# Mountain car example: On-policy control via episodic semi-gradient value approximation

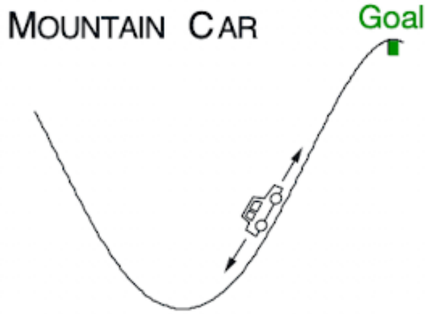



Figure courtesy of Sutton & Barto (2018)

Reward

$R(s) = -1$ for every state $s$, until the car passes the goal at the top right and so ends the episode.

Dynamics $x_{t+1} = \text{bound}(x_t + \dot{x}_{t+1})$

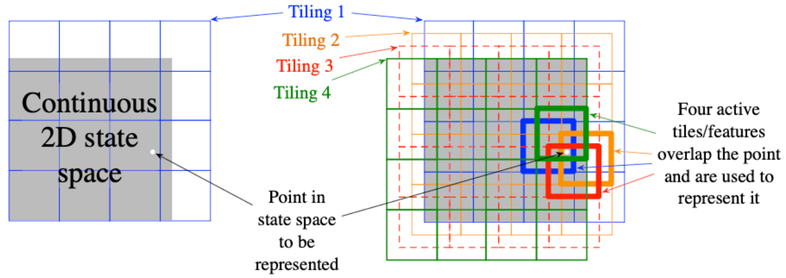
$\dot{x}_{t+1} = \text{bound}(\dot{x}_t + 0.001A_t - 0.0025cos(3x_t))$

Where bound( ) enforces $-1.2 \leq x \leq 0.5$ and $-0.07 \leq \dot{x} \leq 0.07$

Actions -1 (full throttle left), +1 (full throttle right), 0 (do nothing).

Feature construction for value approximation:

Tile coding: represent continuous states $s$ with a sparse, high-dimensional binary feature vector $\phi(s)$ by overlaying multiple grids (tilings) on the state space and marking which tiles the state falls into. Each state activates a small subset of features, which allows linear value function methods to generalize smoothly across nearby states while still distinguishing between them.



# Mountain car example: On-policy control via episodic semi-gradient value approximation

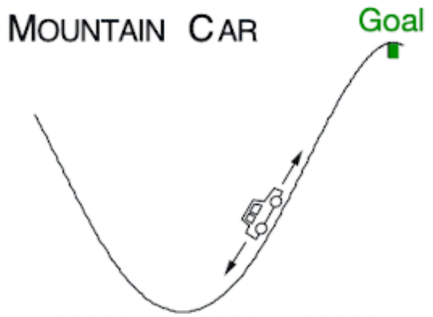



Figure courtesy of Sutton & Barto (2018)

**Reward**

$R(s) = -1$ for every state $s$, until the car passes the goal at the top right and so ends the episode.

**Dynamics**

$$x_{t+1} = \text{bound}(x_t + \dot{x}_{t+1})$$

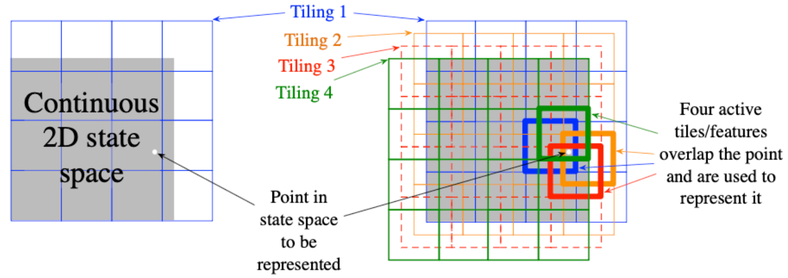
$$\dot{x}_{t+1} = \text{bound}(\dot{x}_t + 0.001A_t - 0.0025cos(3x_t))$$

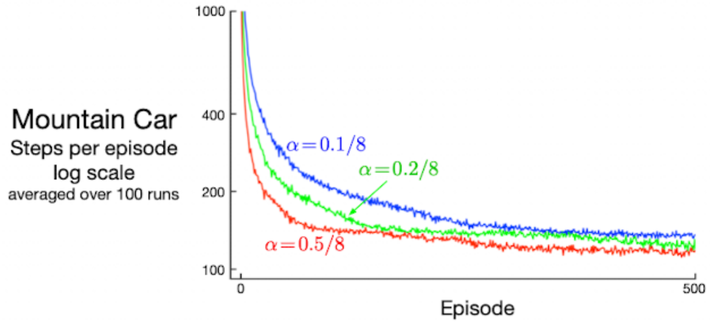
Where bound( ) enforces $-1.2 \leq x \leq 0.5$ and $-0.07 \leq \dot{x} \leq 0.07$

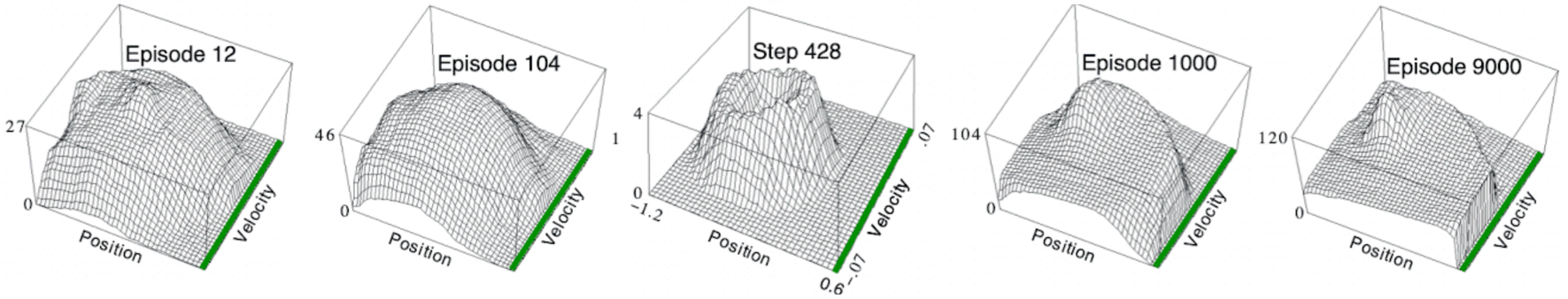
**Actions** -1 (full throttle left), +1 (full throttle right), 0 (do nothing).

**Feature construction for value approximation:**

Tile coding: represent continuous states $s$ with a sparse, high-dimensional binary feature vector $\phi(s)$ by overlaying multiple grids (tilings) on the state space and marking which tiles the state falls into. Each state activates a small subset of features, which allows linear value function methods to generalize smoothly across nearby states while still distinguishing between them.



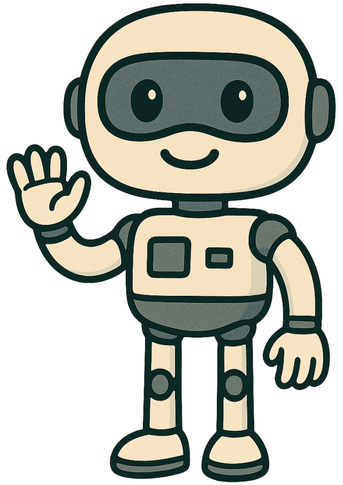




Figures courtesy of Sutton & Barto (2018)

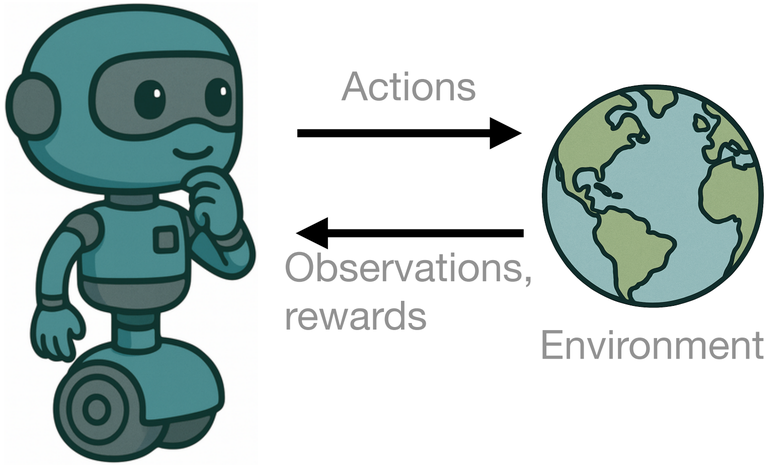
# Recall: Off-policy methods

Objective: Learn $Q_\theta(s, a)$ corresponding to *target policy* $\pi(a \mid s)$, given data generated under a different *behavior policy* $b(a \mid s)$.

Learn a target policy $\pi(a \mid s)$



Using episodes sampled under a different behavioral policy $b(a \mid s)$



# Off-policy methods and value function approximation

Extending off-policy learning methods to include function approximation introduces **significant** complications.

Recall that we previously used *importance sampling* to help use off-policy samples collected under behavioral policy $b(a \mid s)$ to create value estimates $Q_\theta(s, a)$ corresponding to a separate target policy $\pi(a \mid s)$.

# Off-policy methods and value function approximation

Extending off-policy learning methods to include function approximation introduces **significant** complications.

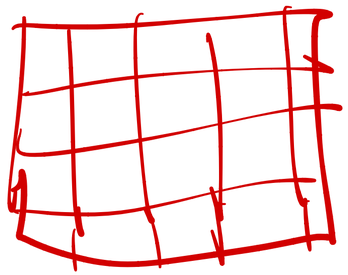
Recall that we previously used *importance sampling* to help use off-policy samples collected under behavioral policy $b(a \mid s)$ to create value estimates $Q_\theta(s, a)$ corresponding to a separate target policy $\pi(a \mid s)$.

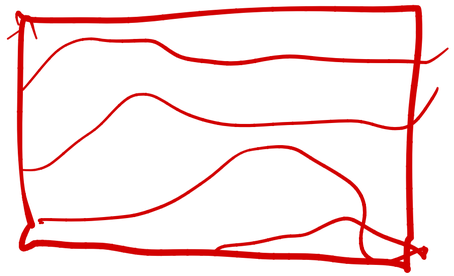
We can similarly use importance sampling for off-policy learning with function approximators by defining the per-step importance sampling ratio:

$$\rho_t = \frac{\pi(A_t \mid S_t)}{b(A_t \mid S_t)}$$

And using this ratio $\rho_t$ to re-weight the individual steps in our SGD update equations.

$$\theta_{t+1} = \theta_t + \alpha\rho_t \left[R_t + \gamma V_\theta(S_{t+1}) - V_\theta(S_t)\right] \nabla_\theta V_\theta(S_t) \qquad \text{or} \qquad \theta_{t+1} = \theta_t + \alpha\rho_t \left[R_t + \gamma Q_\theta(S_{t+1}, A_{t+1}) - Q_\theta(S_t, A_t)\right] \nabla_\theta Q_\theta(S_t, A_t)$$





Using this in conjunction with function approximation can cause the learning process to diverge.

# Off-policy methods and value function approximation

Extending off-policy learning methods to include function approximation introduces **significant** complications.

Recall that we previously used *importance sampling* to help use off-policy samples collected under behavioral policy $b(a \mid s)$ to create value estimates $Q_\theta(s, a)$ corresponding to a separate target policy $\pi(a \mid s)$.

We can similarly use importance sampling for off-policy learning with function approximators by defining the per-step importance sampling ratio:

$$\rho_t = \frac{\pi(A_t \mid S_t)}{b(A_t \mid S_t)}$$

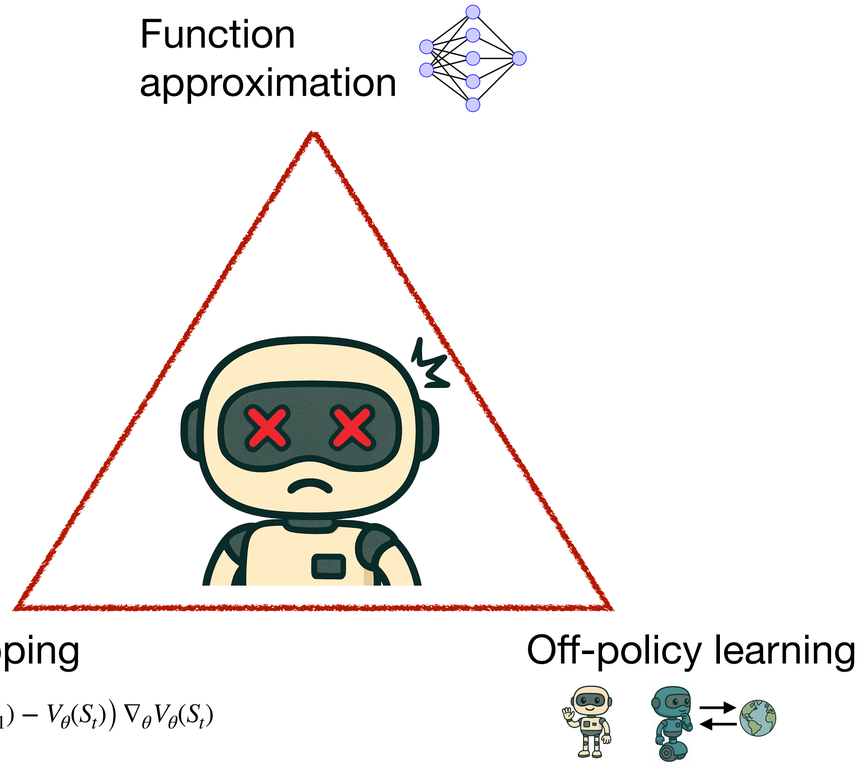
And using this ratio $\rho_t$ to re-weight the individual steps in our SGD update equations.

$$\theta_{t+1} = \theta_t + \alpha\rho_t \left[R_t + \gamma V_\theta(S_{t+1}) - V_\theta(S_t)\right] \nabla_\theta V_\theta(S_t) \qquad \text{or} \qquad \theta_{t+1} = \theta_t + \alpha\rho_t \left[R_t + \gamma Q_\theta(S_{t+1}, A_{t+1}) - Q_\theta(S_t, A_t)\right] \nabla_\theta Q_\theta(S_t, A_t)$$

However, using such techniques in conjunction with function approximation can cause the learning process to diverge!

# The “deadly triad”, and examples of off-policy divergence

Instability in the learning process occurs whenever algorithms employ all three of the following concepts.

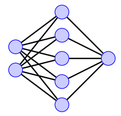



$$\theta_{t+1} = \theta_t + \alpha \left(R_t + \gamma V_\theta(S_{t+1}) - V_\theta(S_t)\right) \nabla_\theta V_\theta(S_t)$$

# The “deadly triad”, and examples of off-policy divergence

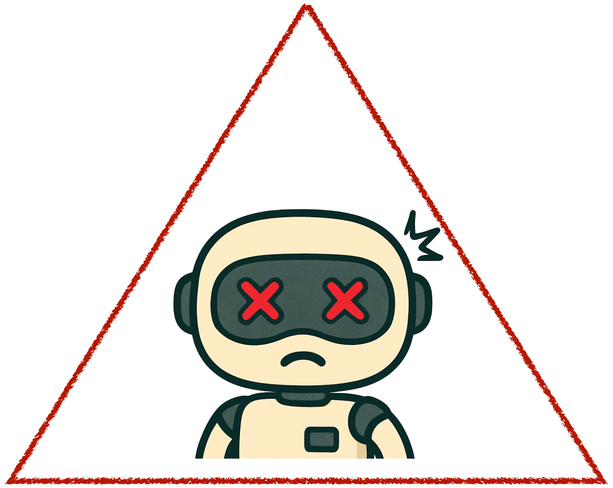
Instability in the learning process occurs whenever algorithms employ all three of the following concepts.

Function approximation



Necessary to scale to large problems with complex features, partial observations, and many states.

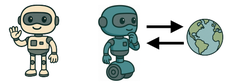
Updates to one state-action pair affect estimates for many other pairs through parameter sharing.



Bootstrapping

$$\theta_{t+1} = \theta_t + \alpha \left(R_t + \gamma V_\theta(S_{t+1}) - V_\theta(S_t)\right) \nabla_\theta V_\theta(S_t)$$

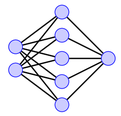
Off-policy learning



# The “deadly triad”, and examples of off-policy divergence

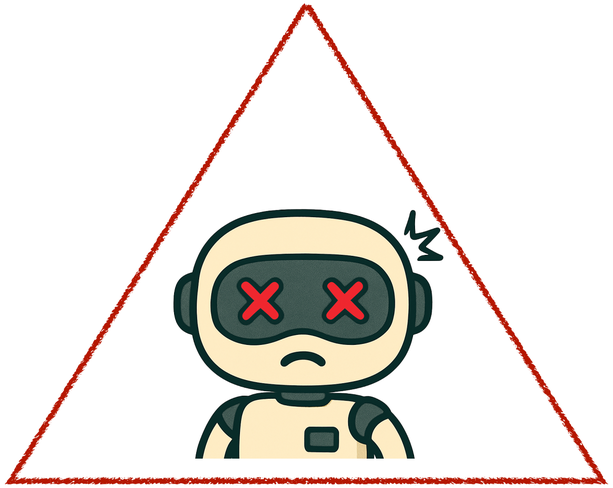
Instability in the learning process occurs whenever algorithms employ all three of the following concepts.

Function approximation



Necessary to scale to large problems with complex features, partial observations, and many states.

Updates to one state-action pair affect estimates for many other pairs through parameter sharing.



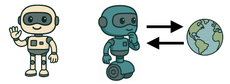
Helps make algorithms computationally efficient, and allows them to learn in continual settings (or before the end of long episodes).

The target itself depends on current, possibly biased estimates.

Bootstrapping

$$\theta_{t+1} = \theta_t + \alpha \left(R_t + \gamma V_\theta(S_{t+1}) - V_\theta(S_t)\right) \nabla_\theta V_\theta(S_t)$$

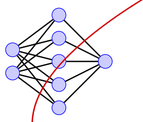
Off-policy learning



# The “deadly triad”, and examples of off-policy divergence

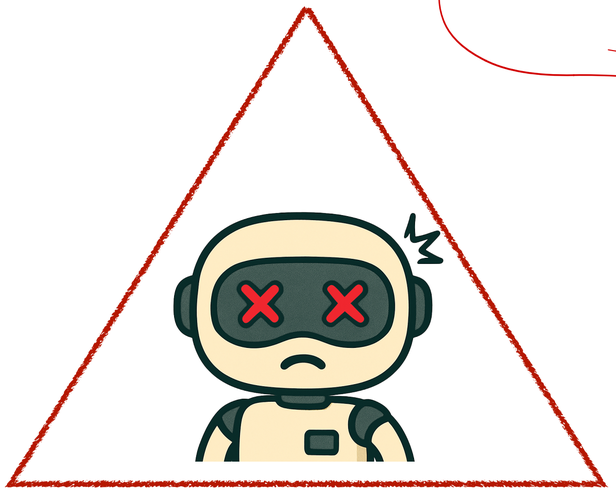
Instability in the learning process occurs whenever algorithms employ all three of the following concepts.

Function approximation



Necessary to scale to large problems with complex features, partial observations, and many states.

Updates to one state-action pair affect estimates for many other pairs through parameter sharing.



Helps make algorithms computationally efficient, and allows them to learn in continual settings (or before the end of long episodes).

The target itself depends on current, possibly biased estimates.

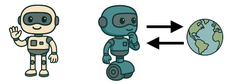
Bootstrapping

$$\theta_{t+1} = \theta_t + \alpha \left(R_t + \gamma V_\theta(S_{t+1}) - V_\theta(S_t)\right) \nabla_\theta V_\theta(S_t)$$

Separate the target policy $\pi$ we are learning from the behavior policy $b$ used to collect data. This is essential for learning many value functions/policies from one continuous stream of experience.

The data distribution (under policy $b$) differs from the distribution we *should* be averagin over (under policy $\pi$).

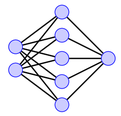
Off-policy learning



# The “deadly triad”, and examples of off-policy divergence

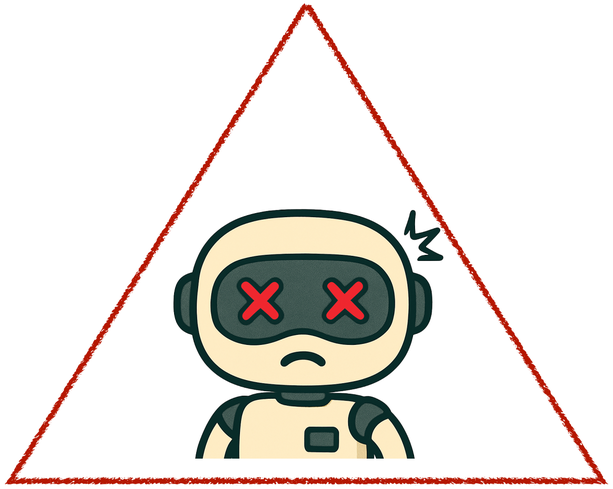
Instability in the learning process occurs whenever algorithms employ all three of the following concepts.

Function approximation



Necessary to scale to large problems with complex features, partial observations, and many states.

Updates to one state-action pair affect estimates for many other pairs through parameter sharing.



Helps make algorithms computationally efficient, and allows them to learn in continual settings (or before the end of long episodes).

The target itself depends on current, possibly biased estimates.

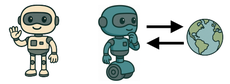
Bootstrapping

$$\theta_{t+1} = \theta_t + \alpha \left(R_t + \gamma V_\theta(S_{t+1}) - V_\theta(S_t)\right) \nabla_\theta V_\theta(S_t)$$

Separate the target policy $\pi$ we are learning from the behavior policy $b$ used to collect data. This is essential for learning many value functions/policies from one continuous stream of experience.

The data distribution (under policy $b$) differs from the distribution we *should* be averagin over (under policy $\pi$).

Off-policy learning



If only one or two of the three elements are present, instability can be avoided.

When all three are present, the effects feedback on each other and the learning process can diverge.

# The “deadly triad”, and examples of off-policy divergence

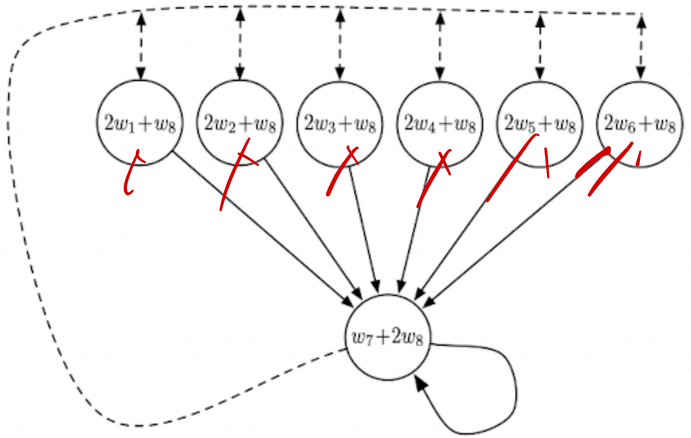
## *Baird’s counterexample:*

True reward/Value function: $R(s) = 0 \implies V^{\pi}(s) = 0, \forall s \in S$

Policy definitions:

Target policy: $\pi(a \mid s) =$ solid, w.p. $1$

Behavior policy: $b(a \mid s) = \begin{cases} \text{dashed, w.p. } \frac{6}{7} \\ \text{solid, w.p. } \frac{1}{7} \end{cases}$




Linear value function approximation:

$V_w(s) = w^T \phi(s)$

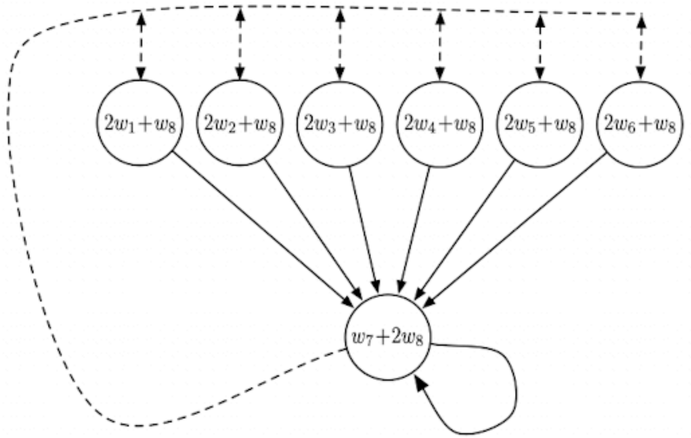
With parameter vector $w \in \mathbb{R}^d$,

And feature vectors $\phi(s) \in \mathbb{R}^d$

e.g., $\phi(s_1) = (2,0,0,0,0,0,0,1)$ for the state $s_1$ on the complete left.

# The “deadly triad”, and examples of off-policy divergence

## *Baird’s counterexample:*




True reward/Value function: $R(s) = 0 \implies V^\pi(s) = 0, \forall s \in S$

Policy definitions:

Target policy: $\pi(a \mid s) =$ solid, w.p. $1$

Behavior policy: $b(a \mid s) = \begin{cases} \text{dashed, w.p. } \frac{6}{7} \\ \text{solid, w.p. } \frac{1}{7} \end{cases}$

Linear value function approximation:

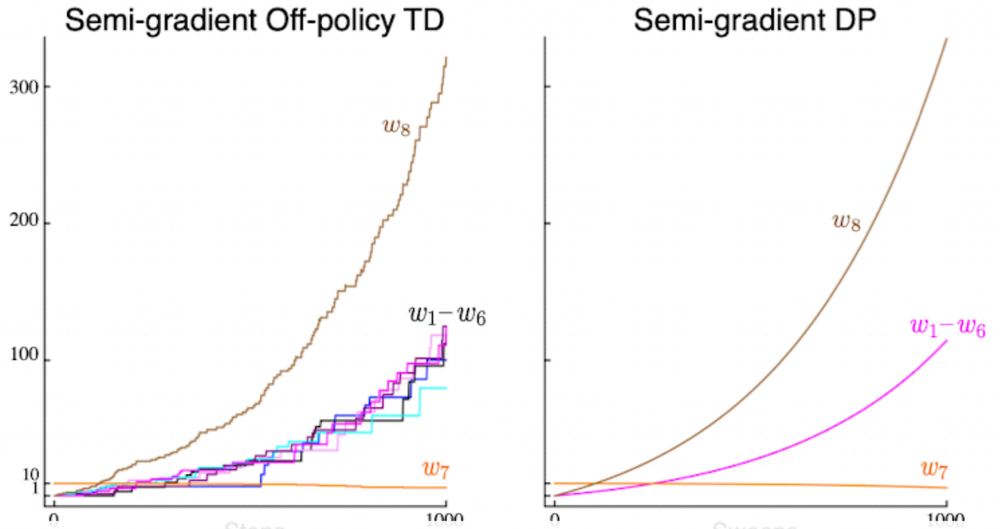
$V_w(s) = w^T \phi(s)$

With parameter vector $w \in \mathbb{R}^d$,

And feature vectors $\phi(s) \in \mathbb{R}^d$

e.g., $\phi(s_1) = (2,0,0,0,0,0,0,1)$ for the state $s_1$ on the complete left.

---



# The “deadly triad”, and examples of off-policy divergence

## *Baird’s counterexample:*

<u>True reward/Value function:</u> $R(s) = 0 \implies V^{\pi}(s) = 0, \forall s \in S$

<u>Policy definitions:</u> Target policy: $\pi(a \mid s) =$ solid, w.p. $1$

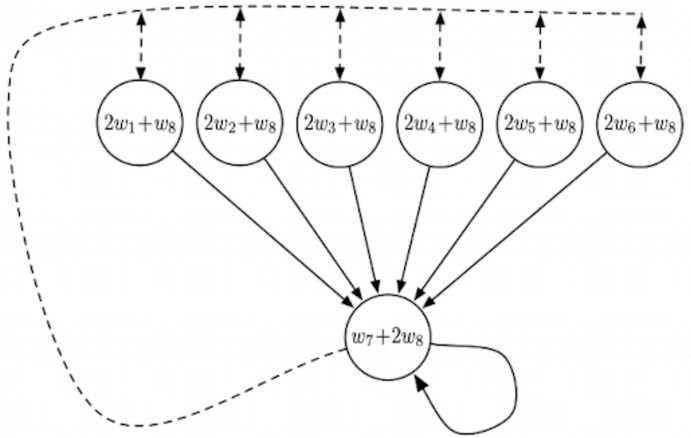
Behavior policy: $b(a \mid s) = \begin{cases} \text{dashed, w.p. } \frac{6}{7} \\ \text{solid, w.p. } \frac{1}{7} \end{cases}$

<u>Linear value function approximation:</u>

$V_w(s) = w^T \phi(s)$

With parameter vector $w \in \mathbb{R}^d$,

And feature vectors $\phi(s) \in \mathbb{R}^d$




e.g., $\phi(s_1) = (2,0,0,0,0,0,0,1)$ for the state $s_1$ on the complete left.

---

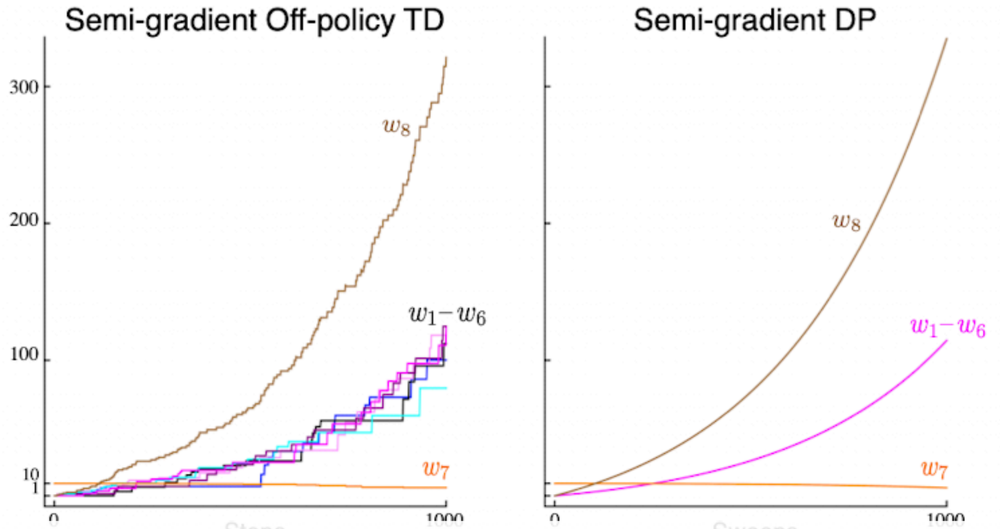



Figure courtesy of Sutton & Barto (2018)

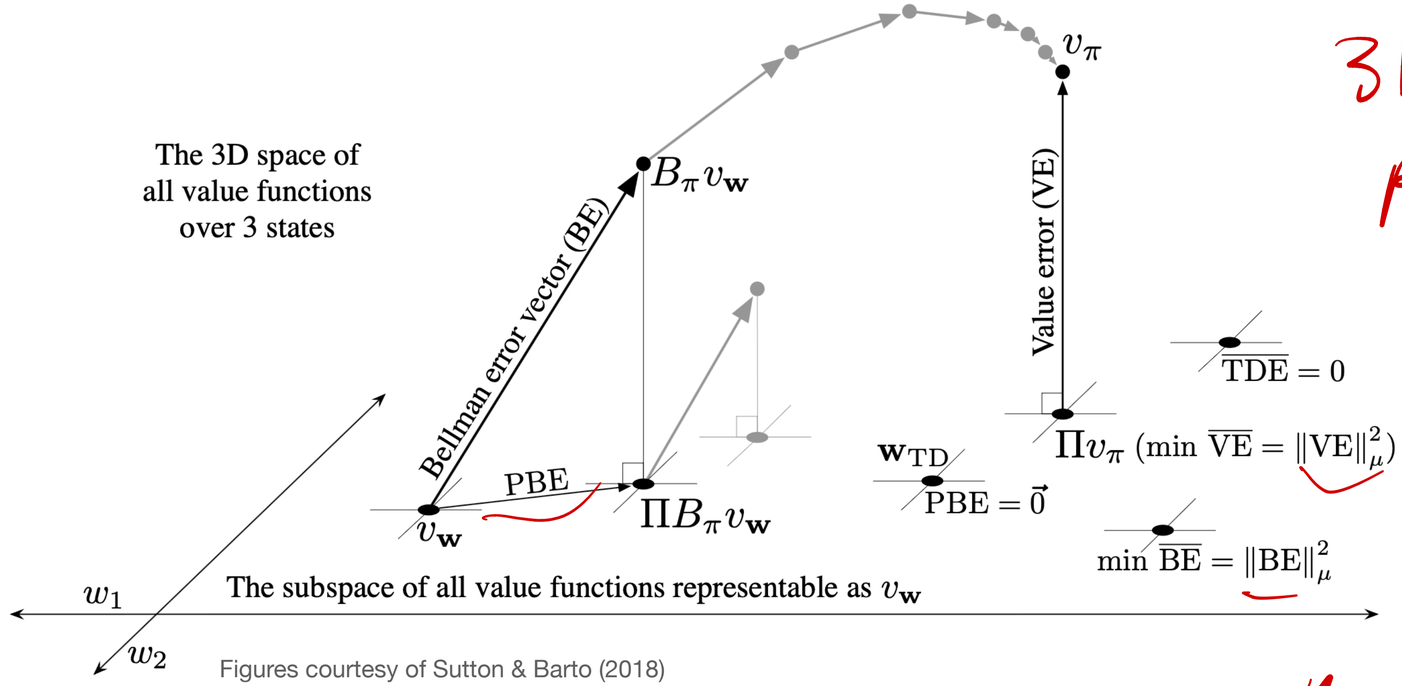
Note: this right-most plot is not even doing TD learning, it is using dynamic programming. The update target is defined by a direct computation of the r.h.s. of Bellman’s equation.

$$w_{t+1} = w_t + \frac{\alpha}{|S|} \sum_{s \in S} \Big( \mathbb{E}\left[ R_t + \gamma V_\theta(S_{t+1}) \mid S_t = s\pi \right] - V_\theta(s) \Big) \nabla_\theta V_\theta(s)$$

And STILL the estimated values are diverging!!

# A visual intuition of value function approximation

Value iteration converges in this 3D space, potentially to a point outside this x-y plane




Figures courtesy of Sutton & Barto (2018)

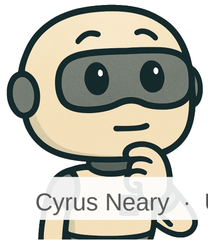
When we do DP, or Q-learning with function approximation:

- Start on the plane
- take a Bellman step off the plane,
- Project back onto the plane, repeat.

This process may converge to a fixed point $\neq \Pi v^\pi$

# How do modern deep RL algorithms deal with these issues?

- Use feature extractors with good inductive biases to avoid correlations between $V_\theta(s)$ for different $s$ that shouldn't exist.
- RL has a non-stationary learning objective. This problem is made worse by bootstrapping.
  - ↳ freeze a "target network" $V^{\bar{\theta}}(s)$ or $Q^{\bar{\theta}}(s,a)$ that we use to define learning targets, and only occasionally update the weights of these frozen networks.
- Experience/replay buffers: Learn from batches of saved data instead of online in a streaming fashion.
  - ↳ Makes it look more like supervised learning.



# Deep Q-Learning

**Playing Atari with Deep Reinforcement Learning**


Volodymyr Mnih  Koray Kavukcuoglu  David Silver  Alex Graves  Ioannis Antonoglou

Daan Wierstra  Martin Riedmiller

DeepMind Technologies

{vlad,koray,david,alex.graves,ioannis,daan,martin.riedmiller} @ deepmind.com


LETTER


doi:10.1038/nature14236


**Human-level control through deep reinforcement learning**


Volodymyr Mnih[1]*, Koray Kavukcuoglu[1]*, David Silver[1]*, Andrei A. Rusu[1], Joel Veness[1], Marc G. Bellemare[1], Alex Graves[1], Martin Riedmiller[1], Andreas K. Fidjeland[1], Georg Ostrovski[1], Stig Petersen[1], Charles Beattie[1], Amir Sadik[1], Ioannis Antonoglou[1], Helen King[1], Dharshan Kumaran[1], Daan Wierstra[1], Shane Legg[1] & Demis Hassabis[1]



**The theory of reinforcement learning provides a normative account[1], deeply rooted in psychological[2] and neuroscientific[3] perspectives on animal behaviour, of how agents may optimize their control of an environment. To use reinforcement learning successfully in situations approaching real-world complexity, however, agents are confronted with a difficult task: they must derive efficient representations of the environment from high-dimensional sensory inputs, and use these to generalize past experience to new situations. Remarkably, humans and other animals seem to solve this problem through a harmonious combination of reinforcement learning and hierarchical sensory processing systems[4,5], the former evidenced by a wealth of neural data revealing notable parallels between the phasic signals emitted by dopaminergic neurons and temporal difference reinforcement learning algorithms[3]. While reinforcement learning agents have achieved some successes in a variety of domains[6–8], their applicability has previously been limited to domains in which useful features can be handcrafted, or to domains with fully observed, low-dimensional state spaces. Here we use recent advances in training deep neural networks[9–11] to develop a novel artificial agent, termed a deep Q-network, that can learn successful policies directly from high-dimensional sensory inputs using end-to-end reinforcement learning. We tested this agent on the challenging domain of classic Atari 2600 games[12]. We demonstrate that the deep Q-network agent, receiving only the pixels and the game score as inputs, was able to surpass the performance of all previous algorithms and achieve a level comparable to that of a professional human games tester across a set of 49 games, using the same algorithm, network architecture and hyperparameters. This work bridges the divide between high-dimensional sensory inputs and actions, resulting in the first artificial agent that is capable of learn-**


agent is to select actions in a fashion that maximizes cumulative future reward. More formally, we use a deep convolutional neural network to approximate the optimal action-value function

$$Q^*(s,a) = \max_\pi \mathbb{E}\left[r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \ldots | s_t = s,\, a_t = a,\, \pi\right],$$

which is the maximum sum of rewards $r_t$ discounted by $\gamma$ at each time-step $t$, achievable by a behaviour policy $\pi = P(a|s)$, after making an observation ($s$) and taking an action ($a$) (see Methods)[19].

Reinforcement learning is known to be unstable or even to diverge when a nonlinear function approximator such as a neural network is used to represent the action-value (also known as $Q$) function[20]. This instability has several causes: the correlations present in the sequence of observations, the fact that small updates to $Q$ may significantly change the policy and therefore change the data distribution, and the correlations between the action-values ($Q$) and the target values $r + \gamma \max_{a'} Q(s', a')$. We address these instabilities with a novel variant of Q-learning, which uses two key ideas. First, we used a biologically inspired mechanism termed experience replay[21–23] that randomizes over the data, thereby removing correlations in the observation sequence and smoothing over changes in the data distribution (see below for details). Second, we used an iterative update that adjusts the action-values ($Q$) towards target values that are only periodically updated, thereby reducing correlations with the target.

While other stable methods exist for training neural networks in the reinforcement learning setting, such as neural fitted Q-iteration[24], these methods involve the repeated training of networks *de novo* on hundreds of iterations. Consequently, these methods, unlike our algorithm, are too inefficient to be used successfully with large neural networks. We parameterize an approximate value function $Q(s,a;\theta_i)$ using the deep

Two landmark papers in deep reinforcement learning.

> Played Atari video games directly from pixel observations, with (super)human-level performance.

These papers implement many of the ideas we've been talking about, with engineering fixes to get around some of the major challenges we discussed.

They were the first to show empirically that Q-learning with neural network function approximators can work very well in practice.

Core idea: use neural networks to approximate $Q_\theta$ directly from pixel observations.

Take gradient steps using a Q-learning inspired semi-gradient update rule.

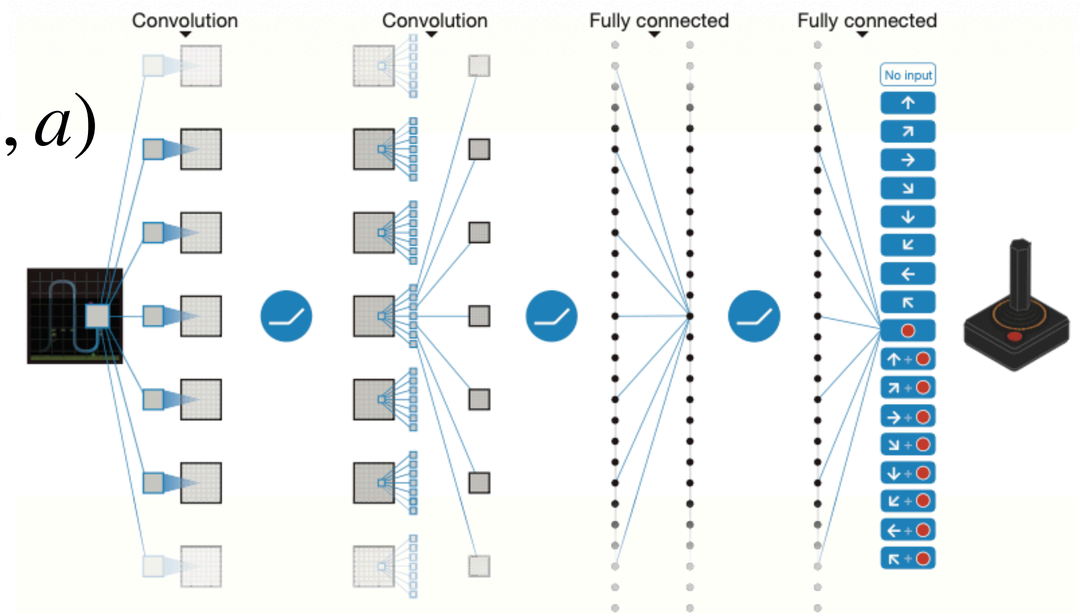
$$\nabla_{\theta_i}(\mathscr{L}(\theta_i)) = \mathbb{E}_{s,a,r,s'}\left[\left(r + \gamma \max_{a' \in A} Q_{\theta_i^-}(s', a') - Q_{\theta_i}(s, a)\right) \nabla_{\theta_i} Q_{\theta_i}(s, a)\right]$$

# Deep Q-learning neural network architecture

Input: History of 4 consecutive processed frames. $s_t = \psi(I_t, I_{t-1}, I_{t-2}, I_{t-3}) \in \mathbb{R}^{84\times84\times4}$.

Here, $I_t \in \mathbb{R}^{210\times160\times3}$ is the RGB image observation at time $t$, and $\psi(\cdot)$ is an image pre-processing function.

$Q_\theta(s, a)$

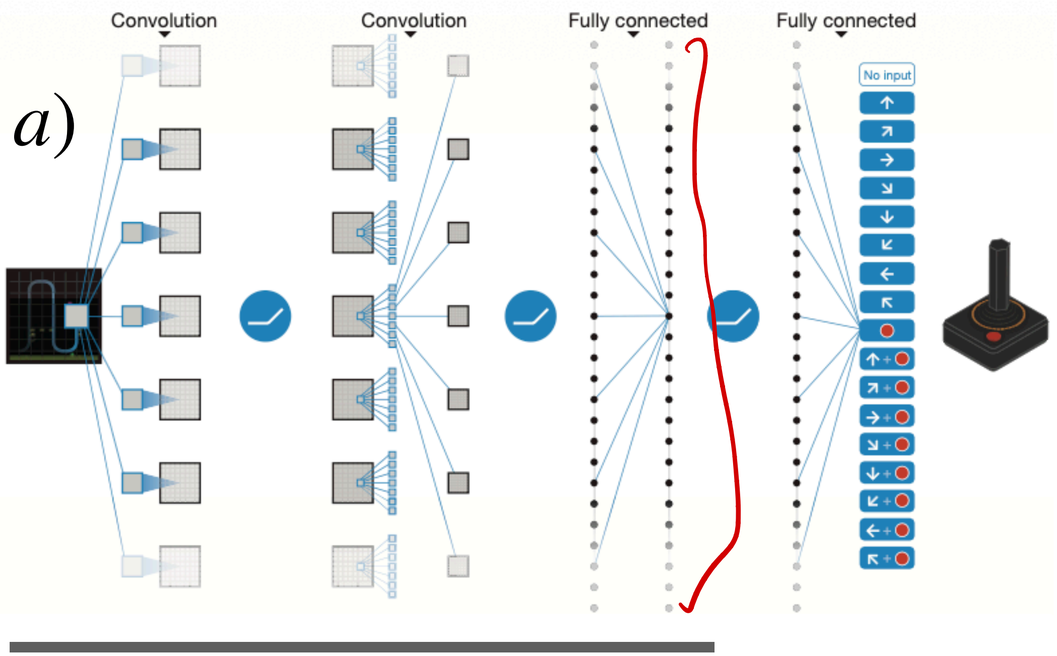


# Deep Q-learning neural network architecture

Input: History of 4 consecutive processed frames. $s_t = \psi(I_t, I_{t-1}, I_{t-2}, I_{t-3}) \in \mathbb{R}^{84\times84\times4}$.

Here, $I_t \in \mathbb{R}^{210\times160\times3}$ is the RGB image observation at time $t$, and $\psi(\cdot)$ is an image pre-processing function.

$Q_\theta(s, a)$



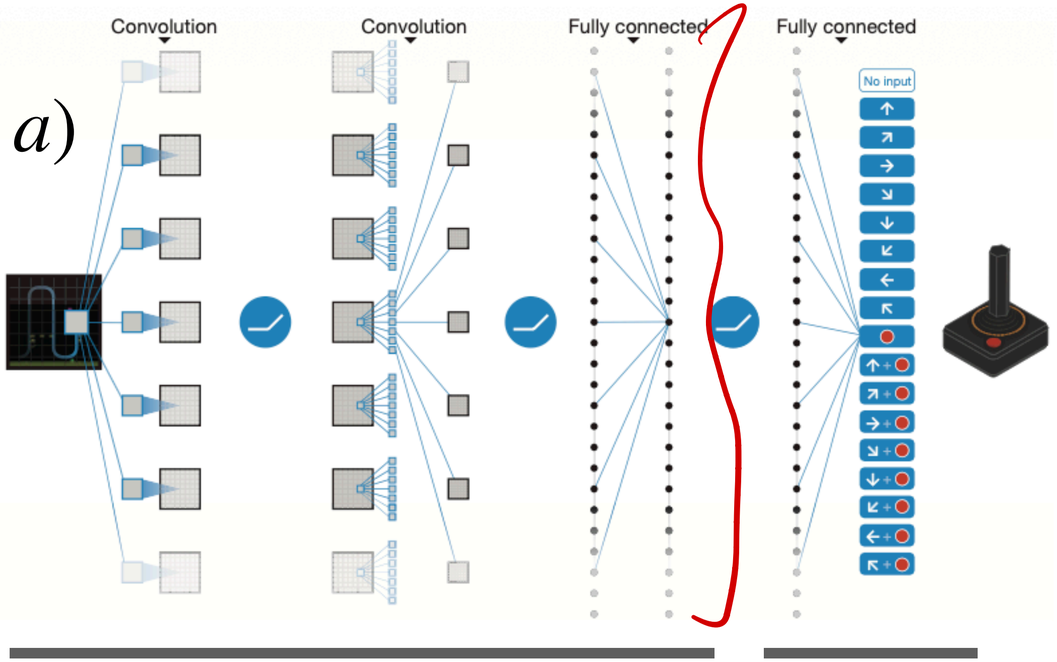

Convolutional + fully connected layer produce latent feature vector $\phi(s_t)$.

# Deep Q-learning neural network architecture

Input: History of 4 consecutive processed frames. $s_t = \psi(I_t, I_{t-1}, I_{t-2}, I_{t-3}) \in \mathbb{R}^{84\times84\times4}$.

Here, $I_t \in \mathbb{R}^{210\times160\times3}$ is the RGB image observation at time $t$, and $\psi(\cdot)$ is an image pre-processing function.

$Q_\theta(s, a)$




Convolutional + fully connected layer produce latent feature vector $\phi(s_t)$.

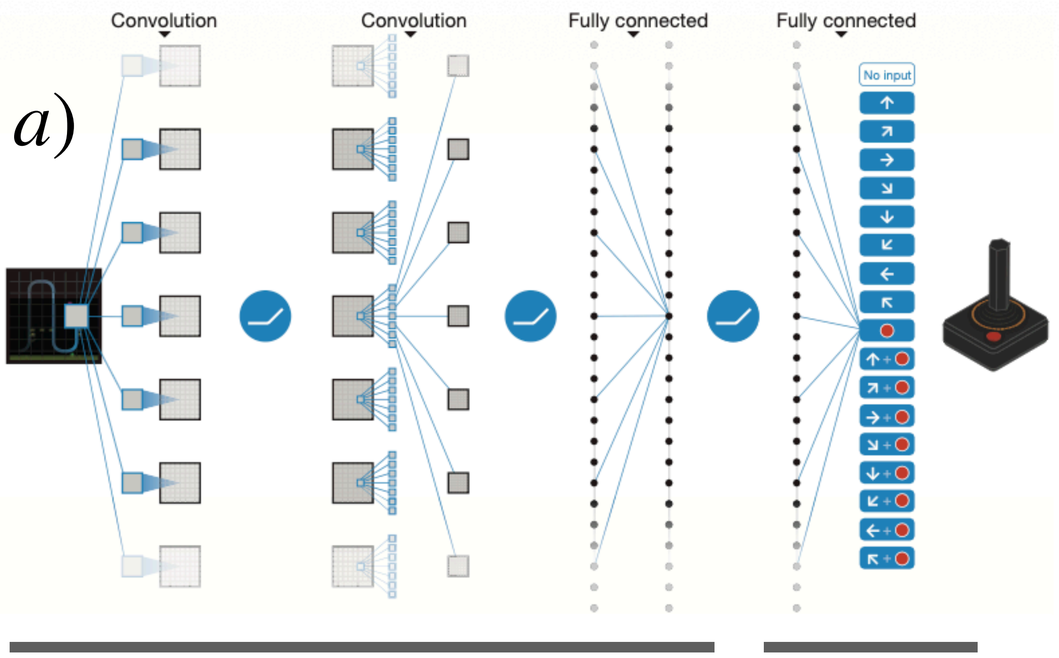
Final linear layer generates separate scores for each action $a \in A$.

# Deep Q-learning neural network architecture

Input: History of 4 consecutive processed frames. $s_t = \psi(I_t, I_{t-1}, I_{t-2}, I_{t-3}) \in \mathbb{R}^{84\times84\times4}$.

Here, $I_t \in \mathbb{R}^{210\times160\times3}$ is the RGB image observation at time $t$, and $\psi(\cdot)$ is an image pre-processing function.

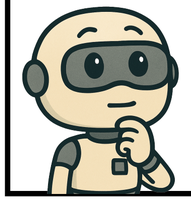
$Q_\theta(s, a)$




Why input the past 4 images?

To help deal with partial observability!

The authors are assuming that a history of 4 frames is enough to convert the POMDP into an MDP from the agent's perspective.



Convolutional + fully connected layer produce latent feature vector $\phi(s_t)$.

Final linear layer generates separate scores for each action $a \in A$.

# Deep Q-learning algorithm

Learn $Q_\theta(s, a)$, an approximation for the optimal state-action value function, by using a semi-gradient method to fit a target defined by Bellman's optimality equations.

**Algorithm 1** Deep Q-learning with Experience Replay

Initialize replay memory $\mathcal{D}$ to capacity $N$
Initialize action-value function $Q$ with random weights
**for** episode $= 1, M$ **do**
  Initialise sequence $s_1 = \{x_1\}$ and preprocessed sequenced $\phi_1 = \phi(s_1)$
  **for** $t = 1, T$ **do**
    With probability $\epsilon$ select a random action $a_t$
    otherwise select $a_t = \max_a Q^*(\phi(s_t), a; \theta)$
    Execute action $a_t$ in emulator and observe reward $r_t$ and image $x_{t+1}$
    Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$
    Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in $\mathcal{D}$
    Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from $\mathcal{D}$
    Set $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$
    Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ according to equation 3
  **end for**
**end for**

Replay
initialize $\theta$
$\epsilon$-greedy exploration
add transition to replay buffer
perform gradient steps on target.
Q-learning target

# Deep Q-learning algorithm

Learn $Q_\theta(s, a)$, an approximation for the optimal state-action value function, by using a semi-gradient method to fit a target defined by Bellman's optimality equations.

**Algorithm 1** Deep Q-learning with Experience Replay

Initialize replay memory $\mathcal{D}$ to capacity $N$
Initialize action-value function $Q$ with random weights
**for** episode $= 1, M$ **do**
  Initialise sequence $s_1 = \{x_1\}$ and preprocessed sequenced $\phi_1 = \phi(s_1)$
  **for** $t = 1, T$ **do**
    With probability $\epsilon$ select a random action $a_t$
    otherwise select $a_t = \max_a Q^*(\phi(s_t), a; \theta)$
    Execute action $a_t$ in emulator and observe reward $r_t$ and image $x_{t+1}$
    Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$
    Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in $\mathcal{D}$
    Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from $\mathcal{D}$
    Set $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$
    Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ according to equation 3
  **end for**
**end for**

For each sampled transition $(s, a, r, s')$:

$$\theta_{t+1} = \theta_t + \alpha \left[ r + \gamma \max_{a' \in A} Q_{\theta_t^-}(s', a') - Q_{\theta_t}(s, a) \right] \nabla_\theta Q_{\theta_t}(s, a)$$

# Deep Q-learning algorithm

Learn $Q_\theta(s, a)$, an approximation for the optimal state-action value function, by using a semi-gradient method to fit a target defined by Bellman's optimality equations.

**Algorithm 1** Deep Q-learning with Experience Replay

Initialize replay memory $\mathcal{D}$ to capacity $N$
Initialize action-value function $Q$ with random weights
**for** episode $= 1, M$ **do**
  Initialise sequence $s_1 = \{x_1\}$ and preprocessed sequenced $\phi_1 = \phi(s_1)$
  **for** $t = 1, T$ **do**
    With probability $\epsilon$ select a random action $a_t$
    otherwise select $a_t = \max_a Q^*(\phi(s_t), a; \theta)$
    Execute action $a_t$ in emulator and observe reward $r_t$ and image $x_{t+1}$
    Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$
    Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in $\mathcal{D}$
    Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from $\mathcal{D}$
    Set $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$
    Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ according to equation 3
  **end for**
**end for**

For each sampled transition $(s, a, r, s')$:

$$\theta_{t+1} = \theta_t + \alpha \left[ r + \gamma \max_{a' \in A} Q_{\theta_t^-}(s', a') - Q_{\theta_t}(s, a) \right] \nabla_\theta Q_{\theta_t}(s, a)$$

Instead of using individual transitions to take gradient steps as they are observed, store the transitions in a *replay buffer* $\mathcal{D}$, and occasionally use the entire replay buffer to define a minibatch gradient step.

# Deep Q-learning algorithm

Learn $Q_\theta(s, a)$, an approximation for the optimal state-action value function, by using a semi-gradient method to fit a target defined by Bellman's optimality equations.

**Algorithm 1** Deep Q-learning with Experience Replay

Initialize replay memory $\mathcal{D}$ to capacity $N$
Initialize action-value function $Q$ with random weights
**for** episode $= 1, M$ **do**
  Initialise sequence $s_1 = \{x_1\}$ and preprocessed sequenced $\phi_1 = \phi(s_1)$
  **for** $t = 1, T$ **do**
    With probability $\epsilon$ select a random action $a_t$
    otherwise select $a_t = \max_a Q^*(\phi(s_t), a; \theta)$
    Execute action $a_t$ in emulator and observe reward $r_t$ and image $x_{t+1}$
    Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$
    Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in $\mathcal{D}$
    Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from $\mathcal{D}$
    Set $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$
    Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ according to equation 3
  **end for**
**end for**

For each sampled transition $(s, a, r, s')$:

$$\theta_{t+1} = \theta_t + \alpha \left[ r + \gamma \max_{a' \in A} Q_{\theta_t^-}(s', a') - Q_{\theta_t}(s, a) \right] \nabla_\theta Q_{\theta_t}(s, a)$$

Instead of using individual transitions to take gradient steps as they are observed, store the transitions in a *replay buffer* $\mathcal{D}$, and occasionally use the entire replay buffer to define a minibatch gradient step.

Use a frozen target network $Q_{\theta_t^-}$ to define the update target $y_j = r_j + \gamma \max_{a' \in A} Q_{\theta_t^-}(s', a')$, instead of differentiating through the target's dependance on $\theta_t$.

# Deep Q-learning algorithm

Learn $Q_\theta(s, a)$, an approximation for the optimal state-action value function, by using a semi-gradient method to fit a target defined by Bellman's optimality equations.

**Algorithm 1** Deep Q-learning with Experience Replay

Initialize replay memory $\mathcal{D}$ to capacity $N$
Initialize action-value function $Q$ with random weights
**for** episode $= 1, M$ **do**
  Initialise sequence $s_1 = \{x_1\}$ and preprocessed sequenced $\phi_1 = \phi(s_1)$
  **for** $t = 1, T$ **do**
    With probability $\epsilon$ select a random action $a_t$
    otherwise select $a_t = \max_a Q^*(\phi(s_t), a; \theta)$
    Execute action $a_t$ in emulator and observe reward $r_t$ and image $x_{t+1}$
    Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$
    Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in $\mathcal{D}$
    Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from $\mathcal{D}$
    Set $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$
    Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ according to equation 3
  **end for**
**end for**

For each sampled transition $(s, a, r, s')$:

$$\theta_{t+1} = \theta_t + \alpha \left[ r + \gamma \max_{a' \in A} Q_{\theta_t^-}(s', a') - Q_{\theta_t}(s, a) \right] \nabla_\theta Q_{\theta_t}(s, a)$$

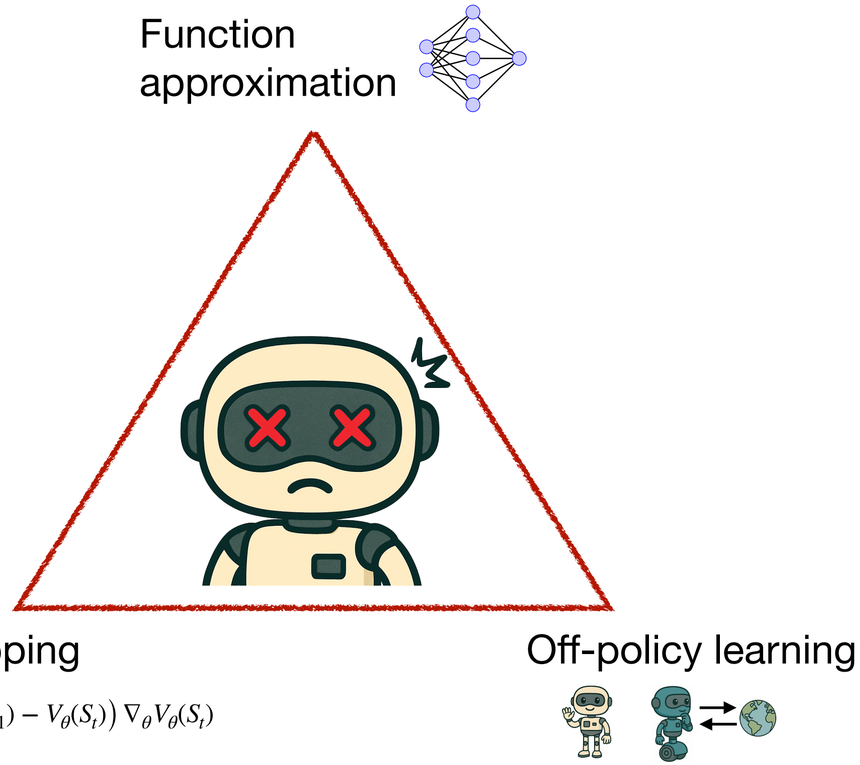
Instead of using individual transitions to take gradient steps as they are observed, store the transitions in a *replay buffer* $\mathcal{D}$, and occasionally use the entire replay buffer to define a minibatch gradient step.

Use a frozen target network $Q_{\theta_t^-}$ to define the update target $y_j = r_j + \gamma \max_{a' \in A} Q_{\theta_t^-}(s', a')$, instead of differentiating through the target's dependance on $\theta_t$.

Otherwise, the core algorithm looks almost identical to tabular Q-learning!

# Engineering tricks to appease the deadly triad

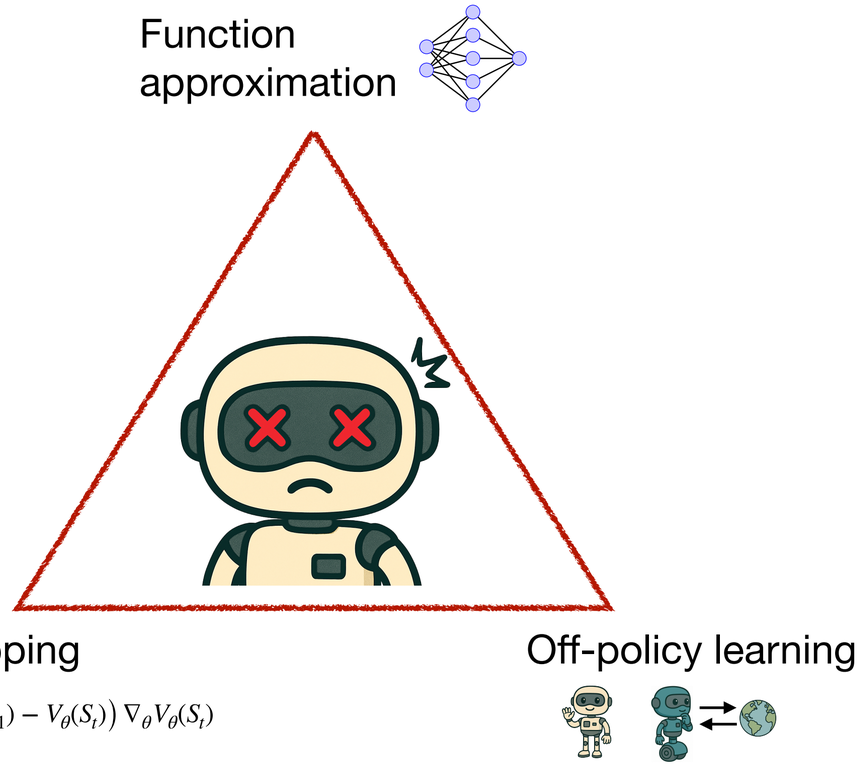
How does this algorithm escape the deadly triad? It introduces engineering tricks to get around these challenges!




$$\theta_{t+1} = \theta_t + \alpha \left( R_t + \gamma V_\theta(S_{t+1}) - V_\theta(S_t) \right) \nabla_\theta V_\theta(S_t)$$

# Engineering tricks to appease the deadly triad

How does this algorithm escape the deadly triad? It introduces engineering tricks to get around these challenges!



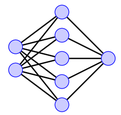

$$\theta_{t+1} = \theta_t + \alpha \left( R_t + \gamma V_\theta(S_{t+1}) - V_\theta(S_t) \right) \nabla_\theta V_\theta(S_t)$$

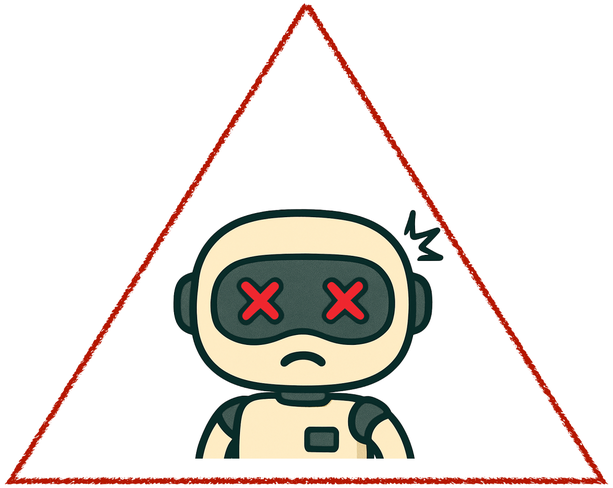
The frozen target network $Q_{\theta_i^-}$ helps stabilize the learning problem.

# Engineering tricks to appease the deadly triad

How does this algorithm escape the deadly triad? It introduces engineering tricks to get around these challenges!

Function approximation



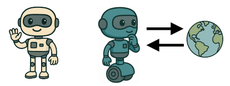


Bootstrapping

$$\theta_{t+1} = \theta_t + \alpha \left(R_t + \gamma V_\theta(S_{t+1}) - V_\theta(S_t)\right) \nabla_\theta V_\theta(S_t)$$

The frozen target network $Q_{\theta_i^-}$ helps stabilize the learning problem.

Off-policy learning

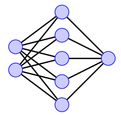


"To alleviate the problems of correlated data and non-stationary distributions, we use an experience replay mechanism which randomly samples previous transitions, and thereby smooths the training distribution over many past behaviors."

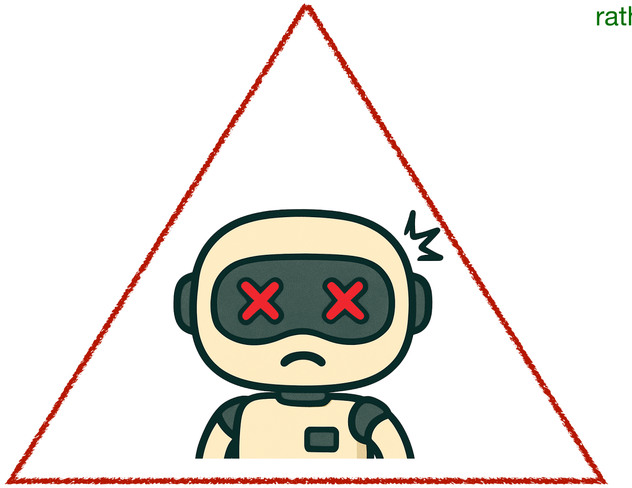
# Engineering tricks to appease the deadly triad

How does this algorithm escape the deadly triad? It introduces engineering tricks to get around these challenges!

Function approximation



State representations (convolution + fully connected embedding) are shared across all actions, so the model's updates for one action in one part of the state space tend to move other predictions in a consistent direction, rather than chaotically.

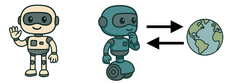


Bootstrapping

$$\theta_{t+1} = \theta_t + \alpha \left(R_t + \gamma V_\theta(S_{t+1}) - V_\theta(S_t)\right) \nabla_\theta V_\theta(S_t)$$

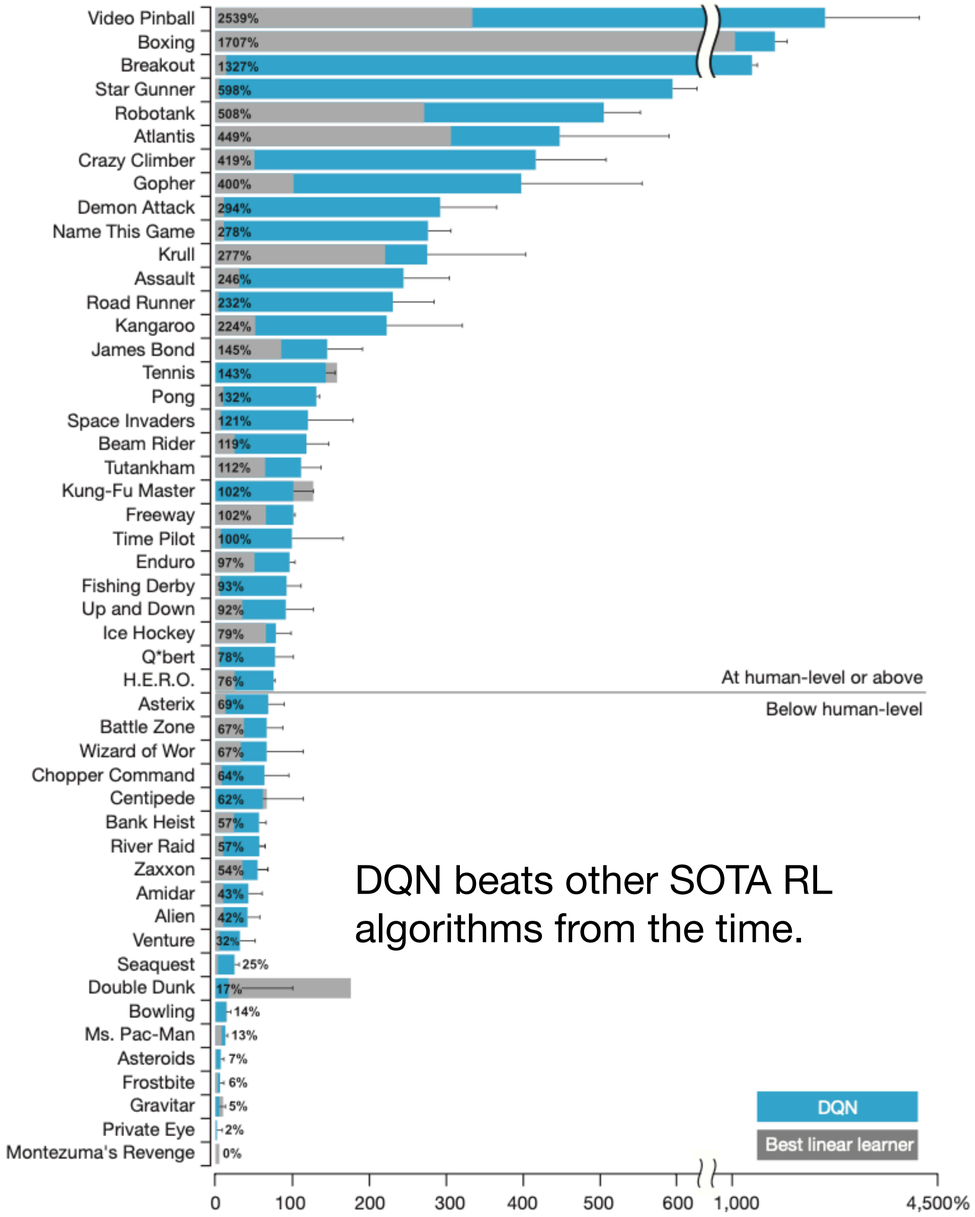
The frozen target network $Q_{\theta_i^-}$ helps stabilize the learning problem.

Off-policy learning

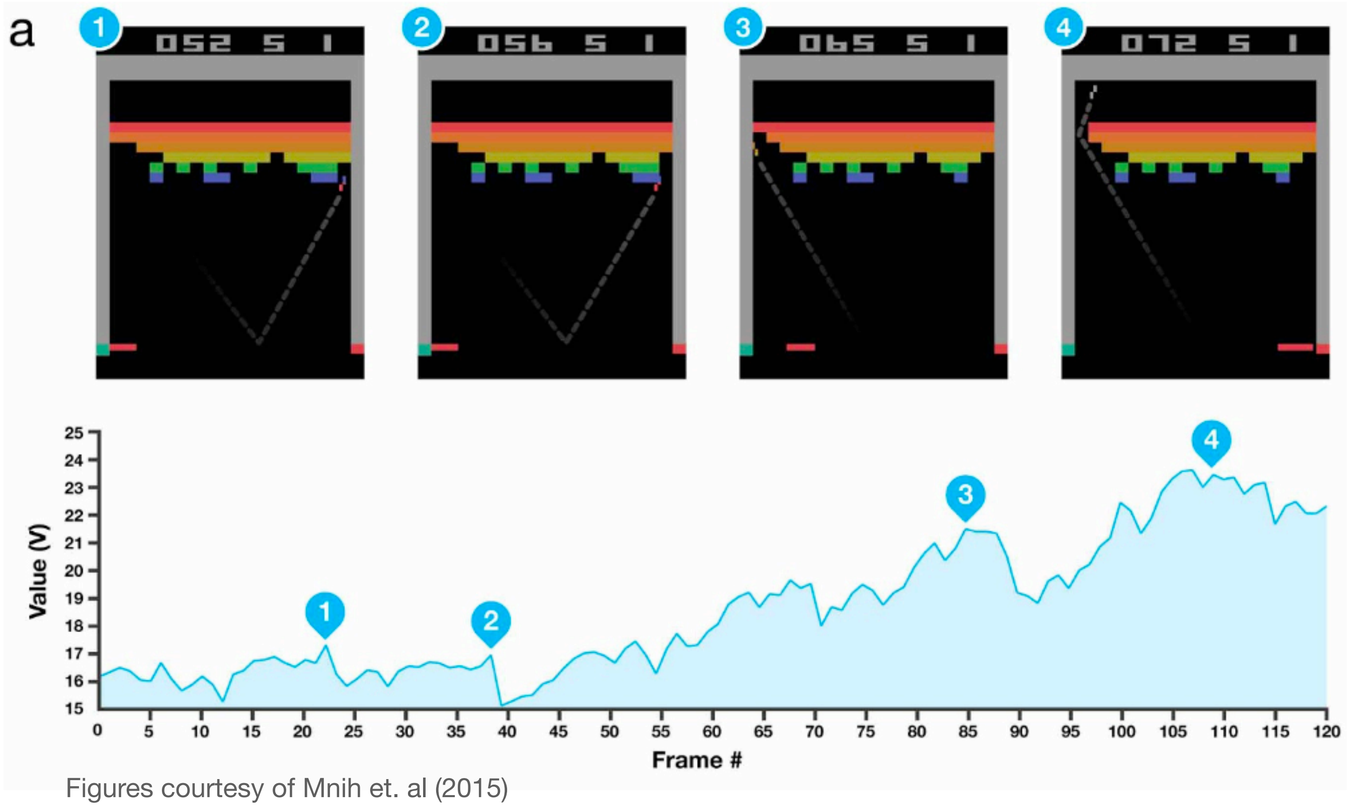


"To alleviate the problems of correlated data and non-stationary distributions, we use an experience replay mechanism which randomly samples previous transitions, and thereby smooths the training distribution over many past behaviors."

# Empirical results




DQN beats other SOTA RL algorithms from the time.




Figures courtesy of Mnih et. al (2015)

- Same algorithm, hyperparameters, and architecture performs across 49 Atari games.
- Reached > human-level performance on many games.
- Stable learning without divergence, despite large (for the time) function approximators.